A Machine Learning Approach to Efficient Simulation-Based Bayesian Estimation of Heterogeneous and Representative Agent Models

Cameron Fen*

October 5, 2023

Heterogeneous agent models are rapidly becoming the workhouse model of modern Macroeconomics but pose estimation challenges compared to their representative agent counterparts. This paper proposes a Machine Learning algorithm for the estimation of structural macroeconomic models. It is a simulation-based algorithm for maximum likelihood or Bayesian estimation, dubbed Sequential Neural Posterior Estimation (SNPE). The approach aims to simulate from the joint distribution of parameters and simulated data, then fits a conditional density estimator on the joint samples of the parameters conditional on the data to get the posterior. This algorithm has the following advantages: estimation is independent of and is unaffected by the choice of solution method or the inner workings of the model, it is faster and more accurate than current benchmark Bayesian algorithms, and simulation-based estimates are statistically efficient when well specified. For example, the posteriors estimated on the benchmark Smets and Wouters (2007) are higher quality and faster compared to Metropolis-Hastings. With Heterogeneous Agent New Keynesian (HANK) models, the estimation algorithm sidesteps lossy dimensionality reduction associated with Reiter's method, and improves estimation speed of models solved via Winberry's method by 13x per iteration. An empirical application estimating a full information heterogeneous agent model with both aggregate time

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Siqi Liu, Xavier Bautista, Florian Gunsilius, David Childers, Eric Jang, John Leahy, and Toni Whited for helpful feedback and Alisdair McKay, Toni Whited, Emil Lakkis, Mohammed Ait Lahcen, and Jeppe Druedahl for code. All errors are my own.

series and cross sectional micro-data demonstrates the capability of this technique. A tutorial can be found at: `https://shorturl.at/pMQ18`

JEL Codes: E27, E37, E47, C45, C11, C15
Keywords: Neural Networks, Bayesian Inference, Dynamic Estimation, Simulation-Based Estimators

I. Introduction

Despite the estimation challenges, macroeconomics is moving more towards the use of heterogeneous agent models due to their increased realism[1]. I introduce a new Machine Learning approach, Sequential Neural Posterior Estimation (SNPE), that can estimate any black box method via simulations in a fast, accurate, and efficient MLE or Bayesian manner. Some problems with heterogeneous agent model estimation include dealing with a high-dimensional state-space, matching both macro and micro data, and reproducing oddly shape, multi-modal, and unidentified posteriors. SNPE uses advances in Machine Learning conditional density estimation to handle the curse of dimensionality, to match real with simulated data, avoiding difficult likelihood functions for micro data, and, in the Bayesian setting, to produce fast, accurate, and reproducible posteriors compared to benchmark procedures. Without much modification, SNPE can estimate models solved via projection, perturbation, value function iteration, Reiter's method (Reiter, 2009), Winberry's method (Winberry, 2018), time iteration (Rendahl, 2017), the Sequence-Space Jacobian approach (Auclert et al., 2021*a*), and many others.

The objective of Bayesian estimation is to derive the posterior distribution, using Bayes' rule to determine the optimal distribution of the parameters, θ, after seeing the data, X. The optimal updating rule (Bayes' rule) is to take the likelihood weighted average of the prior, $P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$. Unlike other algorihtms, SNPE is likelihood free, so by assumption one does not have access to the likelihood, $P(X|\theta)$, but can simulate from it. The algorithm samples from the joint distribution of the parameter space by sampling from the prior and then sampling from the likelihood induced by the prior sample. Then the algorithm uses a Machine Learning conditional density estimator to produce the distribution of the parameters conditional on the the simulated data, $P(\theta|X)$. Conditioning the simulated data on the actual data produces the posterior.

Compared to the baseline Bayesian algorithm, Metropolis-Hastings Markov Chain Monte Carlo (MCMC), SNPE yields better posteriors with less iterations. It is widely known that MCMC is

1. For a review of heterogeneous agent solution methods with which estimation approaches must be compatible, see Reiter (2009), Winberry (2018) and Auclert et al. (2021*a*). Other heterogeneous agent estimation techniques include Ahn et al. (2018), Liu and Plagborg-Møller (2023), and Kase, Melosi and Rottner (2022), but there is more work to be done.

difficult to use and takes a long time to converge (Herbst and Schorfheide, 2015), (Herbst and Schorfheide, 2014). MCMC randomly walks across the posterior with probability equivalent to the joint distribution, in proportion to the posterior. This means MCMC does ultimately sample from the posterior, but takes a long time to converge. This is because the samples exhibit serial correlation, over small time steps, and the sampling procedure has difficulty traversing valleys to alternative modes because the valleys have low probability. At the same time, since SNPE doesn't sample from the posterior with a random walk, but instead learns the posterior via conditional density estimator, SNPE has none of these issues. In fact, theoretical results suggests MCMC converges to multimodal and unidentified posteriors at sub-polynomial rate (Chewi, 2023), while it is likely SNPE is close to quadratic in rate (Farrell, Liang and Misra, 2018), (Draxler, Schnörr and Köthe, 2022).

This matters because the derived posteriors are important for both policy and academic study. Below in Figure I is an example of the marginal distribution of 4 of the 36 parameter Smets and Wouters (2007) DSGE model. The posteriors were evaluated after five hundred thousand iterations of SNPE compared to MCMC after ten million iterations. The other 32 parameters are similar and they are further discribed in Results, Section IV.F. and also in Section BE. of the Appendix. The y axis in the Figure I measures probability mass. The x axis is the support of the posterior defined as ranging from the lower to upper bound of the uniform prior. The red line indicates the true value of the data generating parameter and the blue line is the posterior. The methodology for SNPE and MCMC are described in Sections III.B. and A., respectively.

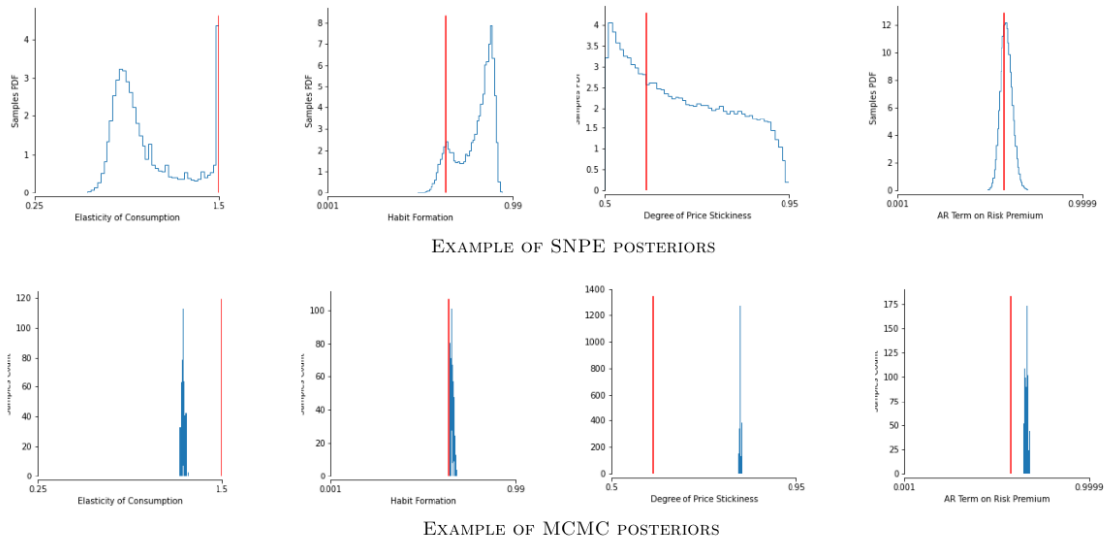

Figure I: Comparison of SNPE Posteriors Compared to MCMC Posteriors

As you can see, SNPE has better coverage as the modes are often at the true parameter value and even when they are not, they always contains the true data generating parameter in the support. Some misidentification of the mode is expected as the model has 36 parameters and was simulated only for 200 timesteps. Additionally this lack of identification underpins many decades of argument about the stickiness of prices and provides a reason why almost every macroeconomist has a different opinion on price stickiness. This seems to suggest that our models, namely Smets Wouters, cannot identify these effects. There are similar results for the other parameters seen in Section IV.F.

An argument could be made that the true posterior is the tighter priors learned by MCMC. However, as one can see, often the MCMC posterior does not contain the actual data generating parameter, suggesting even after 10M iterations, the model is not close to converging and the MCMC posteriors are too tight. Impressively, SNPE gets better results without access to the likelihood function that MCMC uses. When compared to non-likelihood simulation-based approximate Bayesian techniques (Rubin, 1984), SNPE converges to the posterior when the number of iterations goes to infinity (Greenberg, Nonnenmacher and Macke, 2019), while the approximate Bayesian approach does not.

It is less natural, but one can use SNPE as an MLE estimator by using uniform priors and taking the mode of the posterior. When used as a simulation-based MLE point estimator, it also has some advantage over other simulation algorithms. The workhorse simulation algorithm is the Method of Simulated Moments (MSM) (McFadden, 1989). This approach can estimate any model that can be simulated, but is sometimes inefficient even when the model is correctly specified. The estimation algorithm closest to SNPE is the Generative Adversarial Network (GAN) approach of Kaji, Manresa and Pouliot (2020). This algorithm is often efficient with Kaji, Manresa and Pouliot (2020) providing the conditions, but it is slightly different than maximum likelihood. This means that the model likely will produce slightly different parameter values than MLE when misspecified. Kristensen and Shin (2012) also present a similar approach to SNPE, but they use a kernel estimator in their simulated maximum likelihood approach. The kernel approach runs into curse of dimensionality issues as well as problems with conditioning.

Unlike these simulation approaches, SNPE also respects the state-space generating structure of macro models, treating the time-series as a single data point. This is in line with how the the state-space formulation of a solved structural model treats the time series data as a single observation with hundreds of lags. Compared to all the above approaches, my algorithm extends simulation-based approaches to work with Bayesian estimation and can Maximum Likelihood Estimate (MLE) even large models.

In summary, SNPE provides the advantages of full information with simulation-based estimation. Compared to other Bayesian approaches, SNPE produces better posteriors faster and can estimate without knowing the likelihood function. Compared to other simulation-based approaches, SNPE is a full information approach that can estimate even complex models.

II. Literature Review

My paper fits in the intersection of simulation-based structural estimation, Bayesian inference, and the solution and estimation of HANK models. I will talk about simulation-based estimation first. Then I will discuss Bayesian estimation. Finally, I will delve into the solution and estimation

of heterogeneous agent models, including their relevance and current state of the literature. I will cover MCMC procedure in Section III.A. and relevant Machine Learning concepts in section III.B., following the literature review.

II.A. Simulation-Based Estimation in Economics

Simulation-based estimation of models, structural or otherwise, has been a prominent approach in economics for many years. Originating from MSM (McFadden, 1989), simulation-based likelihood and inference techniques have been developed (Pakes and Pollard, 1989), (Duffie and Singleton, 1990) and improved to address the difficulty of model estimation. Maximum Simulated Likelihood (Lerman and Manski, 1981) (Hajivassiliou et al., 1997), (Haan and Uhlendorff, 2006) and other techniques have been used to improve efficiency but still require the use of measurement error. Additionally, Indirect Inference (Gourieroux, Monfort and Renault, 1993) is another simulation-based estimation technique. Researchers have adopted these simulation-based approaches to estimate dynamic models, particularly in Industrial Organization (Keane and Wolpin, 1994), as well as the ever-present usage for structural models in Macroeconomics (Herbst and Schorfheide, 2015) and Finance (Hennessy and Whited, 2007). As a simulation-based technique, SNPE builds upon all these methods.

The issues of MSM have pushed the literature to search for ways to improve robustness to misspecification and efficiency. Even if a model is misspecified, one would like to obtain as narrow standard error bars as possible. Efforts have been made to develop efficient Method of Moment estimators using a spectrum of moments to guarantee that data is in the support of distributions (Gallant and Tauchen, 1996). An additional approach that this paper builds upon is Kaji, Manresa and Pouliot (2020), which allows for efficient estimation of structural models. A detailed comparison with how these models stack up with mine are contained in first table in Table I contained in Section IV.A..

II.B. Bayesian Estimation

The baseline algorithm for Bayesian estimation is MCMC (Herbst and Schorfheide, 2015). However, over the past ten years, there have been improvements in macroeconomic Bayesian estimation. One improvements involved the proposal by Herbst and Schorfheide (2014) to use the Sequential Monte Carlo approach – previously proposed for filtering – for Bayesian estimation. This allows for more effective estimation of multi-modal posteriors and faster convergence. I compare my algorithm to Herbst and Schorfheide (2014) in Table II in Section IV.A. of the Results section.

More recently, Childers et al. (2022) provides a way to differentiate through DSGE models solved via Schmitt-Grohé and Uribe (2004). For the time being, this approach doesn't work for heterogeneous agent models. However, the ability to differentiate through the DSGE model is a powerful tool. Additionally, often more in other fields, the workhorse simulation-based Bayesian estimation approach is Approximate Bayesian Computation (ABC) (Rubin, 1984). Furthermore, there has been some work on estimating macro models with Variational Inference (Loaiza-Maya et al., 2022). However the work has been mainly for estimating reduced-form models like state-space rather than structural models. As mentioned above, a detailed comparison of all these models, with the exception of variational inference, which hasn't been applied to macroeconomic strutural models as far as I'm aware, is contained in Table II in Section IV.A..

II.C. Heterogeneous Agent Models

The literature surrounding full information estimation of heterogeneous agent models is growing and an essential part of the future of macroeconomics. This section will focus on improving speed and accuracy of estimation approaches. The demand for more accurate estimation approaches is significant, given the complexity of the models beings solved. For example, Ahn et al. (2018) proposes to use a Principal Components Analyis (PCA) approach to reduce the dimensionality of the latent space so one can filter, but, as a consequence, information is lost during estimation. Recently there have approaches that combine neural networks projection solution methods along with a complementary estimation approach (Kase, Melosi and Rottner, 2022), which I will discuss

in Section IV.G. of the results.

Another aspect of the Heterogeneous Agent models is the emphasis on computational speed of estimation. There are many papers proposing better solution methods (Auclert et al., 2021*a*), (Winberry, 2018), (Khan and Thomas, 2008), but their complexity means that it is more difficult to estimate with conventional approaches. One innovation on this front is Liu and Plagborg-Møller (2023), which uses full information micro-data to complement macro variable estimation to perform true full-information Bayesian estimation. In particular, they estimate a Krusell and Smith (1998) model via Winberry (2018) in a Bayesian manner in a full information manner on real data. I build upon their approach and discuss advantages and disadvantages of my approach in Section IV.H..

The issues highlighted by estimation – lack of accuracy, computational burden, and identification issues – are reasons so many heterogeneous agent modelers resort to calibration (Auclert et al., 2021*b*), (Ottonello and Winberry, 2020), (Caglio, Darst and Kalemli-Özcan, 2021). There is a need among both heterogeneous agent and representative agent model practitioners to reduce the barriers to MLE and Bayesian estimation, and SNPE and related algorithms can help address these barriers.

III. Background on SNPE and MCMC

In recent years, Bayesian inference has gained increased prominence as an approach for estimating models in economics. This section sheds light on two prominent techniques in this field, namely SNPE, and the benchmark technique for economics, MCMC. To provide a comprehensive understanding, a brief overview of Bayesian estimation is first presented. The SNPE estimator is then discussed.

Despite its success in various applications, the last section will discuss the limitations and pathological cases for the SNPE estimator. While this and related algorithms have been widely used in the Machine Learning literature, it is essential to understand their limitations and make informed decisions when choosing the most appropriate approach for a particular problem.

III.A. Bayesian Basics

In this subsection, I will discuss the general paradigm of Bayesian estimation, followed by a discussion of MCMC.

Bayesian estimation attempts to find the posterior given the likelihood and prior of a model using Bayes' rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

Here $P(\theta|X)$ is the posterior, $P(X|\theta)$ is the likelihood, and $P(\theta)$ is the prior. All approaches that perform Bayesian estimation, ranging from MCMC to variational inference to simulation-based inference, concern techniques for calculating $P(\theta|X)$ without needing to calculate the partition function, $P(X)$.

MCMC is the benchmark technique for Bayesian inference in macroeconomics (Herbst and Schorfheide, 2015). The way MCMC gets around the intractable partition function is to evaluate the likelihood ratios of $P(X|\theta)P(\theta)$ between different parameter values, allowing for the determination of the relative likelihood of different points in the parameter space. As this is a likelihood ratio, one does not need to calculate the normalizing partition function, $P(X)$. This information is then utilized to direct a random walk, with points being visited in proportion to the relative likelihoods, effectively sampling from the posterior distribution $P(\theta|X)$.

One difference between SNPE and MCMC is that the former assumes one cannot calculate the likelihood, $P(X|\theta)$, but only sample from it by sampling from the model. However, the latter assumes that the likelihood can be calculated and MCMC requires that additional assumption for estimation. Despite SNPE being disadvantaged by this assumption, it often produces posteriors faster and more accurate when compared to MCMC, even in cases when the likelihood is tractable.

III.B. Sequential Neural Posterior Estimation (SNPE)

At its core, SNPE is a simulation based approach for Bayesian estimation. The algorithm proceeds in this way: first one draws a sample, θ, from the prior distribution, $P(\theta)$. Subsequently, simulated data, X, is generated from the model likelihood, $P(X|\theta)$. The concatenation of these

two simulations results in samples from the joint distribution, $X, \theta \sim P(X, \theta) = P(X|\theta)P(\theta)$. Note this is equivalent to sampling from the joint distribution numerator in Bayes' rule. These samples can then be used to estimate the posterior distribution, $P(\theta|X)$, through the utilization of either a conditional density estimator like a Conditional Mixture of Gaussians (Bishop, 1994), a Normalizing Flow density estimator (Mohammad-Djafari and Ayasso, 2009) or a GAN (Goodfellow et al., 2014). This section will both discuss the Conditional Mixture of Gaussians and the Normalizing Flow as two examples of density estimators. Finally, conditioning $P(\theta|X)$ on the real data, X', returns an estimate for the posterior: $P(\theta|X = X')$.

The use of more powerful Machine Learning conditional estimators is responsible for much of the improvement over the models proposed in previous literature. Kristensen and Shin (2012) sample from the joint distribution in a similar way as my algorithm. However, they uses a kernel density estimator, which cannot estimate high-dimensional distributions and doesn't allow for Bayesian inference. Likewise Kaji, Manresa and Pouliot (2020) are able to perform something similar to MLE estimation with their GAN approach, but cannot perform Bayesian inference and cannot adhere to the state-space data generating assumption as discussed in Section I.. This is a high level explanation of how SNPE works. However, in reality, there are some addition steps to improve the estimation speed of the algorithm. For example, convergence is quicker if one uses multi-round inference to sample simulated data X in the neighborhood of the real data. Instead of sampling from the prior, θ is sampled from the best approximation of the posterior, leading to more relevant samples of X when it is sampled from the model conditioned on θ. However, this process requires importance sampling to correct for the shift in distribution as θ is no longer being drawn from the prior. The first round entails sampling from the prior, while later rounds involve sampling from the updated estimate of the posterior and applying importance sampling to account for the distribution shift. This can be mathematically expressed as:

$$P'(\theta|X) = P(\theta|X)\frac{P'(\theta)}{P(\theta)} \tag{1}$$

Here $P'(\theta)$ is the proposed distribution that one is sampling from, $P(\theta)$ is the true prior, and $P'(\theta|X)$ is the estimated conditional distribution obtained by sampling from the proposal distribution instead of the prior. The use of a flexible Machine Learning density estimator, such as a Normalizing Flow or a Conditional Mixture of Gaussians (discussed in Section III.E. and Section III.D. respectively), allows for the unbiased estimation of the posterior. This density estimator will be called $P_\phi(\theta|X)$ where the ϕ indicates that this estimator has parameters and P indicates is attempting to approximate the conditional distribution $P(\theta|X)$ by modifying ϕ. When one samples from $P'(\theta)$ instead of the prior $P(\theta)$, the distribution to which ones conditional density estimator converges is $P'(\theta|X)$ from ((1)). If one wants a density estimator to approximate $P(\theta|X)$, $P_\phi(\theta|X)$ will converge to $P(\theta|X)$ if one first adjusts the sample likelihood by the factor $\frac{P'(\theta)}{P(\theta)}$. See Appendix A of Papamakarios and Murray (2016) for a proof and more information. Additionally, Papamakarios and Murray (2016) and Greenberg, Nonnenmacher and Macke (2019) have demonstrated that the use of Universal Approximators as density estimators, such as Normalizing Flows (Huang et al., 2018) and Conditional Mixture of Gaussians (Bishop, 1994) in multi-round SNPE, will converge towards the true Bayesian posterior in the computational limit under the Kullback-Leibler divergence. Below is a description of the SNPE algorithm:

Algorithm 1: SNPE Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', Flow $P_\phi(X|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X, \theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $P_\phi(\theta|X)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{X^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto P_\phi(\theta|X = X')$;

end

One can find the theoretical and empirical advantages SNPE has over other approaches, both Bayesian (Table II) and simulation-based (Table I), in Results, Section IV.A..

III.C. Density Estimation

Roughly, the SNPE approach proposed in this article provides the choice of using one of three distinct density estimators: Normalizing Flows, Conditional Mixture of Gaussians, and GANs. Each has advantages and disadvantages. These methods are capable of handling medium to high-dimensional estimation in X and θ. The structural models are used to generate samples from X and θ, then, with the exception of GANs, which uses the slightly different SNVI algorithm, each density estimator serves as the $P_\phi(\theta|X)$ function in the above Algorithm 1.

One of the primary benefits of the proposed estimation algorithm is the improved handling of conditional, and in some cases high-dimensional, estimation of θ. This allows for Bayesian and MLE estimation of models with up to 36 parameters, such as the Smets and Wouters (2007) model.

This is a significant improvement over previous works, such as Kristensen and Shin (2012), which are limited in their ability to handle high-dimensional and conditional density estimation. This also improves over the GAN approach of Kaji, Manresa and Pouliot (2020) by having a Bayesian extension and the ability to adhere to the state-space data generating process, compared to a cross sectional approach where the time series is chopped up into separate data-points each with only a small number of lags.

Each of the three density estimator approaches has its own strengths and weaknesses. Conditional Mixture of Gaussians is the simplest model to implement and can handle most lower dimensional conditional density estimation with smooth and multi-modal distributions. Normalizing Flows can handle exotic distributions with almost arbitrary dimensionality in θ and X. The GAN approach pared with projecting the derived distribution onto a Normalizing Flow using the reverse Kullback-Leibler divergence, dubbed Sequential Neural Variational Inference (SNVI), results in fast inference combined with the ability to assign no probability to places where sampling fails. SNVI is further discussed in Section CD. of the Appendix.

III.D. Conditional Mixture of Gaussians

The Conditional Mixture of Gaussians approach is a powerful tool for conditional density estimation and, of the three methods, is the easiest to explain. Starting from the unconditional Mixture of Gaussians: as outlined in Bishop (1994), a(n) (unconditional) Mixture of Gaussians is a linear combination of N Gaussian distributions, each with its own mean, covariance, and weight. I will first start by discussing unconditional mixture of Gaussian density estimations. Figure II shows how a Mixture of Gaussians fits a distribution:

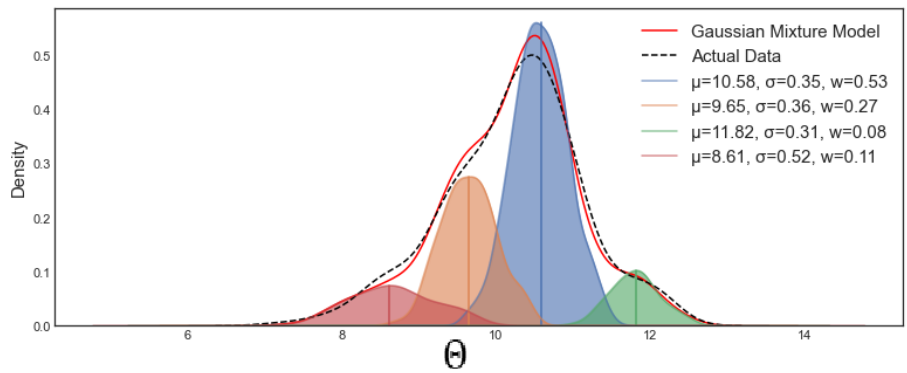

Figure II: How a Gaussian Mixture Model fits a Distribution

The y axis is probability mass and the x axis is parameter value. This is an example of a one-dimensional density estimation for intuition, but the Mixture of Gaussians can handle approximately 10-dimensional density estimation, given my sample size. As one can see, the Gaussian mixture is a weighted average of Gaussians with differing means and standard deviations. By modifying the weights, means and covariance matrices, one can combine a number of Gaussians to accurately represent a distribution. The probability density function (pdf) of a point under this mixture can be calculated using the equation:

$$P(\theta) = \sum_i \pi_i * Q_i(\theta) \tag{2}$$

where $Q_i(\theta)$ is a normal distribution with mean and variance (covariance) specific to each Gaussian component. This represents the unconditional density estimator. However, SNPE requires a conditional density estimator. This can be achieved by using an optimizable function, such as a neural network, that takes the X data as input and returns the parameters for the Conditonal Mixture of Gaussians. If one is unfamiliar with Feed Frward Neural Networks, there is information in the Section AA. of the Appendix. However, for intuition, a high-dimensional polynomial regression or spline would also be able to learn this conditional distribution. The function would then return the weights, $\pi_i(X)$; means, $\mu(X)$; and covariances, $\sigma(X)$, for each Gaussian component as shown

in Equation (3), resulting in the following conditional density estimator:

$$P(\theta|X) = \sum_i \pi_i(X) * Q_i(\theta|\mu(X), \sigma(X)) \tag{3}$$

Finally, I will discuses the estimation procedure. The parameters of the neural network can be optimized via MLE, maximizing $P(\theta|X)$ by adjusting the parameters of the neural network or other function that maps X to the parameters of the Conditional Mixture of Gaussians. The approach maximizes $P(\theta|X)$ given the simulated conditioning data, X, and the Feed Forward Neural Network will learn a mapping from X to the parameters of the Conditional Mixture of Gaussians (i.e. $\sigma(X)$, $\mu(X)$ etc.). In theory, this should imply that the Conditional Mixture of Gaussians will converge to the true conditional distribution.

The problem of this approach is that it runs into curse of dimensionality quickly. For example, Smets and Wouters (2007) has 36 parameters. With a Conditional Mixture of Gaussians, each covariance matrix of a Gaussian would have dimensionality $37 * 36/2 = 666$. Thus for higher dimensionality problems, one will either have to use Normalizing Flows, as I generally do (and explain next), or a GAN, which is discussed in Section CB. of the Appendix.

III.E. Normalizing Flows

I will frame this section discussing a particular type of Normalizing Flow, the Neural Autoregressive Flow (Huang et al., 2018). Then I will discuss how to use the change of variables formula to learn a mapping from realizations of a random variable to samples from a target distribution one wants to estimate. Using the change of variables, one can derive a likelihood of a sample under the Normalizing Flow, allowing one to ultimately perform likelihood estimation to perform density estimation. I will also attempt to highlight advantages of Normalizing Flows have over Kernel Density Estimators (KDEs) that make the SNPE algorithm more powerful than the traditional Kristensen and Shin (2012) approach. The advantages include the ability to conditional density estimate, high-dimensional density estimate, and MLE estimate without resorting to rejection sampling.

At a high level, a Normalizing Flow is a mapping from known distribution samples – for example, a multivariate normal, $N(\bar{0}, \bar{1})$ – to samples from a distribution you want to density estimate. The mapping is a composition of many invertible change of variables such that the output of one change of variables becomes the input of the next change of variables. For example f is a change of variables so the composition stack would be $f_{K-1} \circ f_{K-2} \circ ...f_0$. If we call the samples from the known distribution: z_0, we have: $z_K = f_{K-1} \circ f_{K-2} \circ ...f_0(z_0)$. Now since all these functions are change of variables, samples from a distribution z'_K, which we want z_K to match, have a valid likelihood under the Normalizing Flow. Then by modifying the parameters of each of the change of variables we can maximize the likelihood of samples z'_K occurring under the Normalizing Flow.

Given the intuitive explanation, let me discuss the math in more detail. Starting again, take a function f_k which takes in realizations, z_k, from a random variable and produces samples z_{k+1}. The formula for $P(z_{k+1} = f_k(z_k))$ given $Q(z_k)$ is the familiar change of variables:

$$P(z_{k+1} = f_k(z_k)) = Q(z_k = f_i^{-1}(z_{k+1})|det\frac{df_k^{-1}(z_{k+1})}{dz_{k+1}})| = Q(z_k)|det\frac{df_k(z_k)}{dz_k}|^{-1} \tag{4}$$

Given a series of transformations $z_K = f_{K-1} \circ f_{K-2} \circ ...f_0(z_0)$, then the formula for the log likelihood of the whole sequence of change of variables is:

$$ln(P(z_K)) = ln(Q(z_0)) - \sum_{k=0}^{k=K-1} ln(|det\frac{df_k(z_k)}{dz_k}|) \tag{5}$$

Notice how we can sample $z'_K \sim P(z'_K)$, samples from the distribution we want to match, then equate it with z_K, the output of the Normalizing Flow. Then evaluate the likelihood of z'_K under the Normalizing Flow. In this way, after optimizing the Normalizing Flow, the distribution of z_K points will match the distribution of the points z'_K. This process though, requires the inverse process as the likelihood of a point z'_K is dependent on calculating z_0. Thus one has to be able to invert the Normalizing Flow in addition to being able to sample from it via the forward process. This means that each f_k must be an invertible function (i.e., a bijection) which was assumed earlier.

Definitionally, each f_k is called a bijector and a chain of f's in conjunction with the realizations of the initial known base distributions, z_0, is called a Normalizing Flow. Given this information, we can create a flexible function that can convert realizations from a simple distribution, for example a multivariate normal with mean vector $\bar{\mathbf{0}}$ and standard deviation vector $\bar{\mathbf{1}}$, into realizations of many different "non-parametric" distributions. Figure III illustrates how the Normalizing Flow can transform the realizations of a 2-dimensional normal random variable to a distribution whose scatter-plot spells the word BRAD:

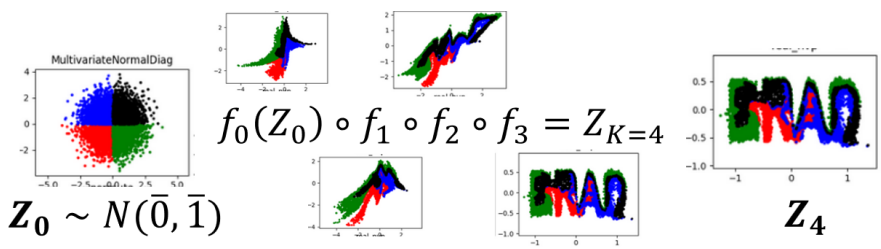

Figure III: Normalizing Flow: Transforming Realizations From a 2-D Normal, into a Distribution Whose Scatter-plot Spells BRAD

The x and y axis of all the pictures in the BRAD flow example illustrate the realizations of sample points along both axes of each z_k intermediate transformation as the Normalizing Flow maps realizations from a multivariate normal to the realizations of a distribution that spells BRAD. z_0 is a 2-dimensional Gaussian and thus because the Normalizing Flow is invertible, each intermediate z_k up to the output z_K are two dimensional. Likewise the picture that correspond with each f_k (alternating above and below) illustrate the slow transformation of the normal noise into the BRAD scatter-plot at the end. This is a much more difficult distribution to estimate than SNPE posteriors which have continuous, full, and connected support, and if they are multi-modal, in a much more straightforward way than this BRAD distribution. However, theoretical results support the flexibility of Normalizing Flows. Huang et al. (2018) – the model this paper uses – shows that by chaining together f_k's as a Normalizing Flow, under some assumptions, one can map to realizations from any continuous distribution.

The next section deals with invertibility of f_k. f_k is a J dimensional vector-valued function that

takes in the J dimensional z_k vector and returns a J dimensional z_{k+1} vector. Some syntax: A subscript under a variable refers to the layer in the Flow. For example, the z_k is the input into the kth f function among all the f's ($f_0 ... f_{K-1}$) that are composed with one another. Additionally there are capital K f's (K = 0 to K-1) and K+1 z's (K = 0 to K). The output of each f_k is z_{k+1} which is the the input to the next f, f_{k+1}. As both z_k and f_k are vectors/vector-valued functions, the superscript, j, indicates the element j of the vector z_k, or f_k, with, as implied earlier, J being the size of the vector z_k.

A simple but inflexible function that is a bijection is an affine transformation:

$$z_{k+1} = a_k * z_k + b_k \tag{6}$$

and its inverse:

$$z_k = \frac{z_{k+1} - b_k}{a_k} \tag{7}$$

If one does not include nonlinear relationships, linear combinations of Gaussians are still Gaussian and so the flow has limited ability to approximate flexible distributions. However, Equation (8) presents a three element f_k function that has conditionality and more flexibility:

(8)

$$\begin{aligned} z^1_{k+1} &= a^1_k * z^1_k + b^1_k \\ z^2_{k+1} &= a^2_k(z^1_k) * z^2_k + b^2_k(z^1_k) \\ z^3_{k+1} &= a^3_k(z^1_k, z^2_k) * z^3_k + b^3_k(z^1_k, z^2_k) \\ &\ldots \end{aligned}$$

The important thing to note is the function on the first element of z_k is still affine. However, the

a_k^j and b_k^j terms, with j greater than 1, are functions, that take in z terms and output coefficients. The function on the second element a_k^2 and b_k^2 are dependent on z_k^1. Likewise, the constants of the third line of Equation (8) is dependent on z_k^1 and z_k^2. Additional equations follow this same pattern. These a_k and b_k functions are Feed Forward Neural Networks, which are discussed in Section AA. in the Appendix. However, for intuition, these function could be high-dimensional polynomial regressions and the parameters to optimize would be the linear βs.

This structure allows the Normalizing Flow to remain invertible. To see this, first assume one knows all the capital J elements of z_k. Since the right hand side is entirely a function of all J elements of the z_k vector, one can calculate all the terms in z_{k+1}.

The reverse is more complicated. Suppose one knows all the elements of z_{k+1} and needs to find the values of z_k. Since z_{k+1}^1 is known, one can invert the first line of Equation (8), the affine equation ($z_{k+1}^1 = a_k^1 * z_k^1 + b_k^1$) to get the value of z_k^1. Now that z_k^1 is known, a_k^2 and b_k^2 only take in z_k^1, one can invert the second line of Equation (8): $z_{k+1}^2 = a_k^2(z_k^1) * z_k^2 + b_k^2(z_k^1)$ to get z_k^2 from z_{k+1}^2 and z_k^1. Now that z_k^1 and z_k^2 are known one can recover z_k^3 from the third line of Equation (8) and so on.

One can show, with slight additional modifications, that the Flow described here, a rough approximation of the Neural Auto-regressive Flow, allows SNPE and related algorithms to converge under the Kullbeck-Leibler divergence to the Bayesian posterior (Greenberg, Nonnenmacher and Macke, 2019). Similar results are true for GANs and Conditional Mixture of Gaussians (Papamakarios and Murray, 2016).

III.F. SNPE Theory

Proposition 1: Mixture of Gaussians are a universal approximator of continuous distributions.

See Barron (1993) for the proof. See `https://mjt.cs.illinois.edu/dlt/#classical-approximations-and-universa` for an explanation of how the proof extends to a Mixture of Gaussians.

Proposition 2: A Neural Auto-regressive Normalizing Flow with some regularity conditions (i.e.,

for example having Sigmoid and inverse Sigmoid non-linearity) is a universal approximator of continuous distributions.

See Appendix F and G of Huang et al. (2018).

Proposition 3: SNPE algorithm will converge to the true posterior if the posterior is contained in the parameterization of the posterior conditional density estimator.

See Papamakarios and Murray (2016) for proof of Proposition 3 and note it is an application of the consistency of MLE estimator applied to the posterior conditional density estimator.

III.G. Caveats and Pathological Examples Using SNPE

There are certain limitations associated with SNPE that must be considered when applying it to a specific problem. One of the limitations of SNPE is its inability to handle the discontinuities that are present in some models. This leads to a lack of smoothness in the posterior of the kinked models, and the assumption that there is a Normalizing Flow parameterization that can generate the posterior is violated. In practice, a Normalizing Flow still converges, but it learns a continuous function that approximates the discontinuity in the posterior. This approximation can still be near the true likelihood, but if estimation with a discontinuous model is a problem, one can also add measurement error to smooth the problem out.

Another limitation of SNPE is its difficulty in handling stochastic singularities, which also result in estimation issues. When this occurs, the dynamic model learns a lower dimensional sub-manifold in the entire space. By assumption, the Normalizing Flow or Conditional Mixture of Gaussians has to have positive support on the entire space of the target distribution. As in the case of the BRAD scatter-plot, the biggest problem isn't having regions of zero support, but having a support that is a lower dimensional manifold of the proposed space. In this case the Normalizing Flow will have to learn a "knife's edge" distribution which will be difficult. In cases with these stochastic singularities

in a model estimated on real data, the model will fail to match the data. However, one can add more shocks, add measurement error, or make a change in modeling to avoid a stochastic singularity (Caballero and Engel, 2007) and allow full support over the range of the model output.

III.H. Summary

SNPE is an algorithm that attempts to perform Bayesian estimation without utilizing a likelihood function. Instead, the approach samples from the joint distribution: $X, \theta \sim P(X, \theta) = P(X|\theta)P(\theta)$. Once enough joint samples are produced, a conditional density estimation is estimated on the samples identifying the conditional distribution $P(\theta|X)$. Conditioning the generic X on the real data X' gets the posterior. The advantage my approach is mostly due to the improvement in the conditional density estimators. Normalizing Flows and, to a lesser extent, Mixture of Gaussians can handle higher dimensionality and conditional density estimations, which previous approaches like kernel density estimation have difficulty with. Because of this SNPE converges faster and does not require likelihood evaluations compared to baseline MCMC techniques.

IV. Results

This section will evaluate the performance of SNPE by examining the proper behavior of the posterior and inspecting its accuracy. To this end, seven models are estimated using the SNPE algorithm, each showcasing a unique aspect and characteristic of the algorithm. The first model I will walk through my approach with, is the Real Business Cycle (RBC) model. The second model is a toy corporate finance model that is solved via value function iteration. The third model is the Lucas Asset Pricing Model solved through projection. The fourth model is a bequests model solved via value function iteration. These four models are discussed in more detail in Section B of the Appendix. The fifth model is the Smets-Wouters model, which is estimated through both the MCMC and SNPE algorithms. The sixth model is the HANK model solved through Reiter's method and time iteration. The seventh model improves on full information estimation of a Winberry (2018) model by speeding up the approach taken by Liu and Plagborg-Møller (2023).

This model is extended to an empirical application estimating both micro and macro moments in a full information setting. The details of the construction and implementation of these models can be found in Section BG. of the Appendix.

In the empirical analysis, the accuracy of the posterior is inspected by comparing it with the calibrated parameters producing the simulated data. Through the Bernstein von Mises Theorem, the posterior should converge to the true values, although the structural models may not satisfy the assumptions of the theorem. Although the correct posterior should converge to the true parameter value as the number of data samples increases, finite sample sizes will not result in a perfect match. Hence, it is important to assess the posterior mode and ensure that it generally matches the true value and that there is no vanishing posterior mass at the true values. This check provides useful information to determine the performance of SNPE in practice and how it compares with other estimation techniques, such as the MCMC algorithm. While in many cases SNPE is tested using simulated data, in order to resemble real world data, 200 time steps are sampled from the simulated model, to maintain a rough correspondence to the length of real world macroeconomic data. This small number of time steps implies that some parameters will be inadequately identified especially in the models with a large number of parameters. For the sake of illustrating the estimation benefits, all priors will be uniform over the range specified in the graphs. This most effectively demonstrate the ability of the estimation algorithm to converge on a posterior without guidance from informative priors. First, I will provide some charts demonstrating the capabilities of SNPE compared to simulation based algorithms and then Bayesian algorithms.

IV.A. Adaptability of SNPE to Estimation of Models Solved via a Variety of Methods

In order to depict the advantages of SNPE, I provide charts depicting the capabilities of different models. As described in the intro, SNPE can perform simulated maximum likelihood estimation (SMLE) by taking the mode of the posterior with flat priors. Additionally it can perform Bayesian estimation. Thus I have two tables: one comparing SNPE to other simulation-based algorithms and another comparing SNPE to Bayesian approaches. These charts will hopefully illustrate, whether

you want to do frequentist simulation-based estimation or Bayesian estimation, SNPE will be a strong choice as the algorithm to use.

Here is the legend:

Symbol	Meaning
✓	Model can do this without difficulties
$\sim$	Model can often come close to but not find the exact solution
?	Could not find research
✗	Model cannot do this

First, comparisons of simulation-based algorithms:

TABLE I: Comparisons of Simulation-based Algorithms

Solution Method/Algorithm	**SNPE**	**MSM**	**Indirect**	**K & S**	**GAN**
Estimate small scale DSGE	✓	✓	✓	✓	✓
Estimate medium/large DSGE	✓	✓	✓	✗	✓
Estimate HANK - Reiter	✓	✓	✓	✗	✓
Estimate HA - Winberry	✓	✓	✓	✗	✓
Perform SMLE	✓	✗	✗	✓	$\sim$
Adhere State-Space DGP	✓	✗	$\sim$	✗	✗

Here K & S indicates Kristensen and Shin (2012) and GAN indicates Kaji, Manresa and Pouliot (2020). The first two criteria are the ability to estimate a small and then medium/large DSGE model. The third and forth criteria are the ability to estimate HA models in two different ways. The fifth critereon is the ability to perform simulation based MLE and the last criterion is the ability to adhere to the state-space data generating process for structural models. As mentioned earlier,

a serious problem with these models is that they do not respect the state-space data generating process (DGP) of structural models (criterion six). Instead, in order for these approaches to properly estimate, they add a cross-sectional dimension, where each time step in question has a small number of lagged data and each data point is a separate time step with these lags. This would be the proper way to estimate a VAR model, but because solved structural models have a state-space structure, they should be estimated with the entire time series as a single data point. These simulation-based algorithms do not have Bayesian extensions. MSM can estimate any model like SNPE, however it is not always efficient even when the model is will specified. Likewise for Indirect Inference. The GAN approach can estimate all kinds of structural models, is efficient when the model is correctly specified, but doesn't quite perform MLE estimation. An advantage of the GAN approach is that, as a purely frequentist approach, it is usually faster than SNPE, when frequentist estimation is all that is required. The only technique that is simulation-based and also Bayesian is the Approximate Bayesian Computation (ABC) technique of Rubin (1984). I choose to display the ABC approach in the Bayesian algorithms table below (Table II of this section) rather than the simulation-based algorithm section. Nevertheless, it's unsurprising that SNPE seems to be more flexible and powerful compared to competing simulation-based estimation algorithms.

The second table is a comparison of Bayesian approaches:

TABLE II: Comparisons of Bayesian Algorithms

Solution Method/Algorithm	**SNPE**	**MCMC**	**HMC**	**SMC-est**	**ABC**
Estimate small scale DSGE	✓	✓	✓	✓	∼
Estimate medium/large DSGE	✓	✓	✓	✓	∼
Estimate HANK - Reiter	✓	∼	✗	∼	∼
Estimate HA - Winberry	✓	✓	✗	?	∼
Estimate without likelihood function	✓	∼	✓	∼	✓
Quality posteriors even with large models	✓	✗	∼	✓	∼

In addition to SNPE and the benchmark MCMC, I compare with the Hamiltonian Monte Carlo (HMC) approach of Childers et al. (2022), the SMC estimation of Herbst and Schorfheide (2014), and ABC (Rubin, 1984). The first four criteria are the same as for the previous table. The fifth criterion is the ability to do Bayesian estimation from simulation only without using a likelihood function. The final criterion is to have quality/fast posteriors even with large models. MCMC suffers from the inability to converge with large models. The HMC approach has not been applied to Heterogeneous Agent models. Meanwhile SMC works for most of the criteria, but can't be estimated without a likelihood function, has issues with hyper-parameter tuning, and is difficult to manage. ABC is a simulation-based Bayesian technique, but only converges to an approximate posterior. In summary, the SNPE is generally more flexible and powerful than other baseline techniques. Next, I will discuss the individual results of SNPE model by model.

IV.B. RBC Model

I will walk through the RBC model using my algorithm. First, as with all of my models, I use flat posteriors defined over an appropriate range. In this case, the parameters – α, captial share; β, discount rate; δ, depreciation rate; and ρ, productivity persistence parameter – all range from 0 to 1. After sampling parameters from this four-way uniform distribution, I sample simulated data from my RBC model, conditioned on the parameters sampled from the prior. The RBC model produces simulated data: productivity, interest rate, capital, output and consumption. The joint distribution of the sampled parameters and the simulated data is used to train a conditional density estimator that learns the pdf $P(\theta|X)$, where θ is the simulated parameters, and X is the simulated data. Then conditioning X on the real data, X', one learns the posterior: $P(\theta|X = X')$. This is a simplified intuitive application of the SNPE algorithm. As a result of applying SNPE, Figure IV is the posterior distribution. More graphs and information is provided in Section BA. of the Appendix.

In the below image (IV), the graphs along the main diagonal, indicate the marginal distributions of the posteriors. The y axis on the diagonal graphs illustrates increasing probability mass and the x axis indicates the support of the posterior which ranges from the lower bound to the upper bound

of the uniform prior. The off-diagonal charts are two way contour plots where the x axis corresponds the the marginal distribution at the same vertical axis and the y axis corresponds to the marginal at the same horizontal axis. The green and yellow colors on this plot indicate increasing probability mass and the blue indicates little or no probability mass.

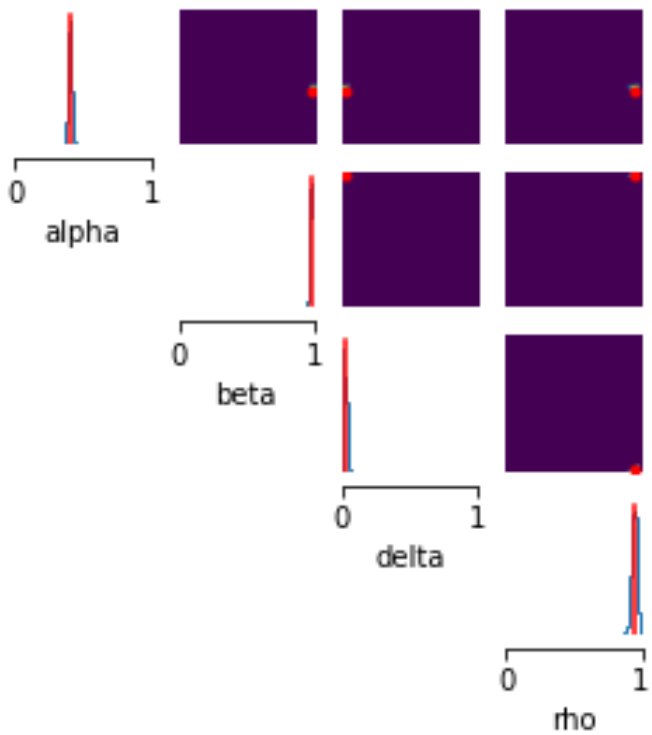

Figure IV: Simulated Data RBC

As you can see, the posterior mode concentrates on the real parameter value, suggesting convergence.

IV.C. Cash Flow Model

Figure V is the posterior picture that comes from estimation of a cash flow model:

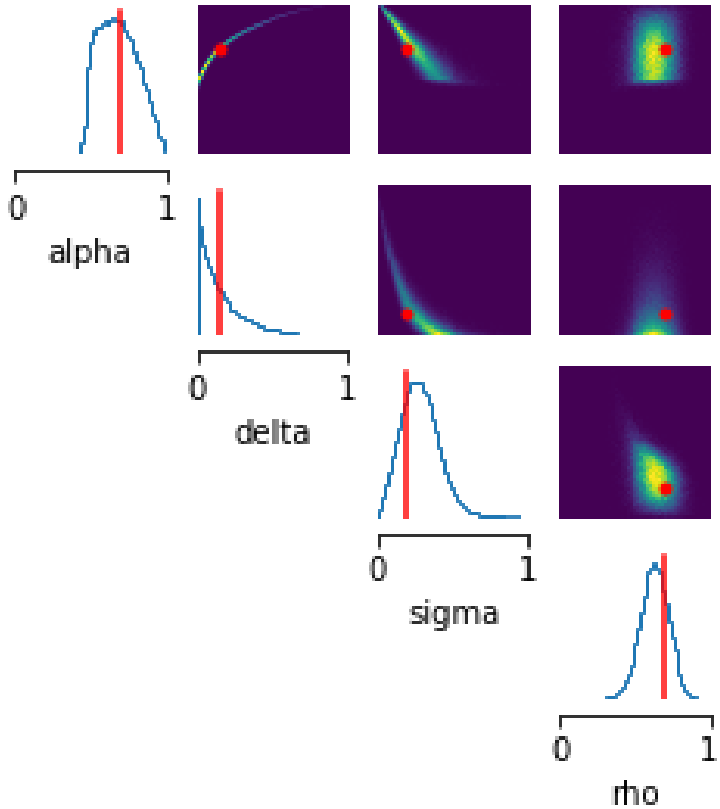

Figure V: SNPE Estimation of a Cash Flow Model Solved via Value Function Iteration

The modes concentrate on the true parameter value, suggesting convergence. More details are found in the Section BB. of the Appendix.

IV.D. Projection - Lucas Tree

Figure VI is the posterior picture that comes from the Lucas Tree model

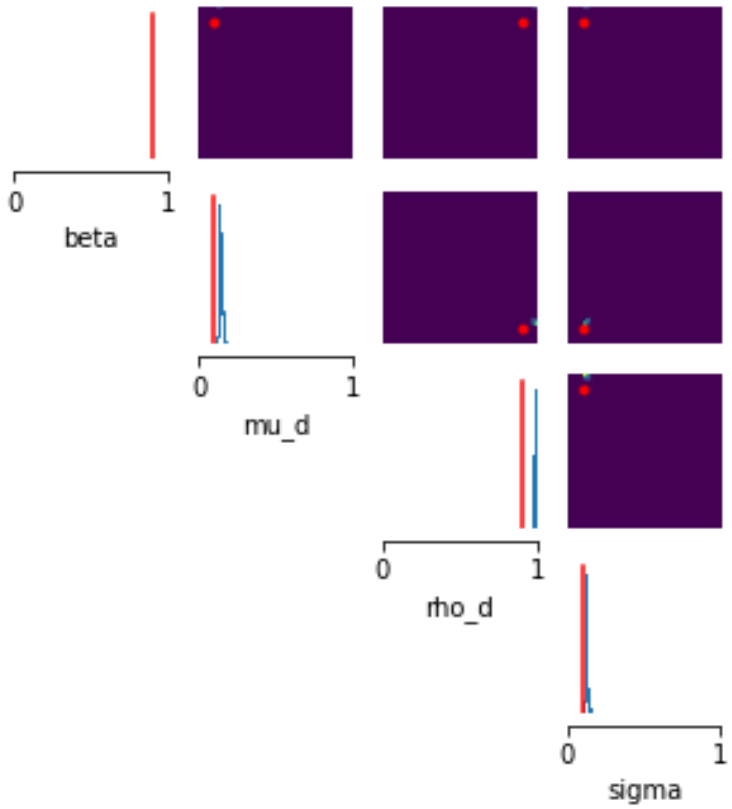

Figure VI: Lucas Tree Projection Model

Again, the modes concentrate on the true parameter value, suggesting convergence. More details are found in Section BC. of the Appendix.

IV.E. Value Function Iteration - Bequests Model

The fourth model considered in this study is an 11-parameter Bequests Model. Figure VII corresponds to the parameters in discussed further Section BD. of the Appendix.

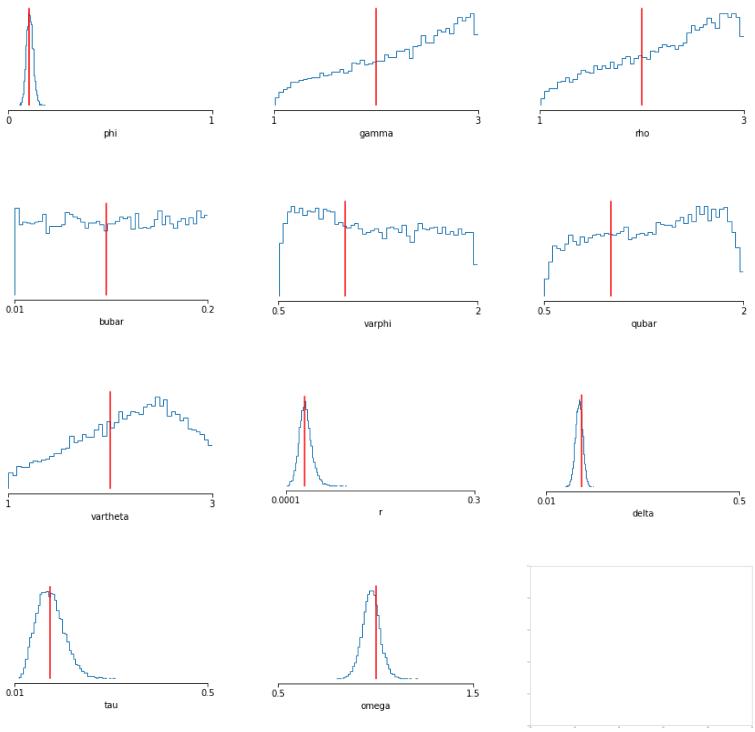

Figure VII: Eleven Parameter Bequest Model Estimated via VFI

The results of the estimation of the model parameters via Value Function Iteration (VFI) are shown in Figure VII. Although many of the parameters have converged to the true values, some of the parameters still exhibit high uncertainty. This uncertainty is due to the fact that the data was only 200 time steps long. Many of the posterior marginal distributions do concentrate around the true values, suggesting that the model has some discerning power with regard to the parameters.

Next I will discuss the main results estimating the Smets-Wouters 2007 model as well as one HANK model and one Heterogeneous agent non-HANK empirical application.

IV.F. Smets-Wouters 2007 Model

The Smets-Wouters (2007) DSGE model is a Representative Agent New Keynesian model that describes an economy with a flexible price sector and a sector with sticky Calvo prices. The model has eight observed variables and eight shocks that interact with the rest of the model in an auto-

regressive manner. The linearized form of the model has 36 parameters. SNPE estimated the posterior using roughly 500,000 samples generated from the prior and likelihood. The parameters are separated into three different charts of 12 parameters each, and only marginals are displayed to avoid visual clutter. Like the diagonal of the previous charts, the y axis indicates probability mass and the x axis indicates posterior support. These graphs are histograms rather than smoothed with kernel. This illustrates that the density estimators, be they Conditional Mixture of Gaussians or Normalizing Flows, generate natural looking posteriors, which is not something MCMC generates. The variable labels are difficult to read, due to the effort to provide a reasonable number of parameters in each page. Section BE. of the Appendix has a list of variables in the order they are displayed. Only the first 12 parameters are displayed to reduce clutter. The rest are displayed in the end of Section BE. of the Appendix.

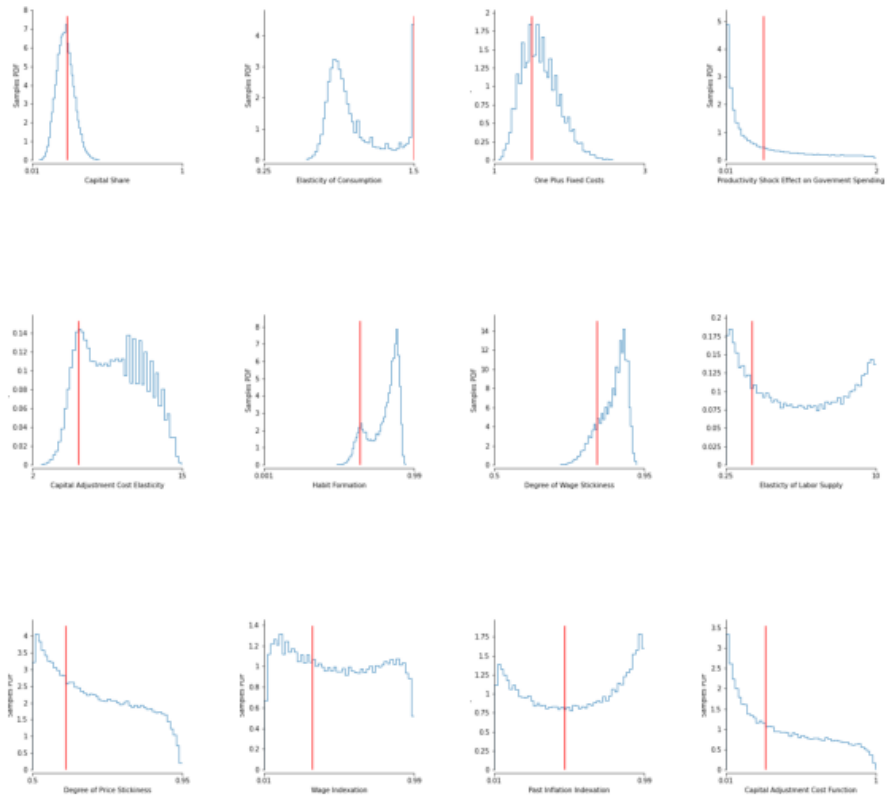

Figure VIII: SNPE Smets-Wouters Posterior with 500 Thousand Samples: Parameters 1-12,

The Smets-Wouters model, estimated with the SNPE algorithm, demonstrates varying levels of success in estimating the various parameters. For example, the model accurately estimates the capital share but has more difficulty with the adjustments costs. The (one plus) fixed costs are estimated well. You can also see identification of multimodal parameters like habit formation and elasticity of consumption. In accordance with the difficultly of indentification in the literature, price stickiness is quite flat.

The posteriors estimated by the Smets-Wouters model with MCMC are less successful in covering the true data generating process parameters. Both MCMC and SNPE algorithms used the Smets-Wouters Dynare (Adjemian et al., 2022) file to solve the model and much of the MCMC used the Dynare file for estimation with a few modifications (uniform priors, for instance) Despite this, the

MCMC algorithm took roughly 30 hours to draw 10 million samples, and ultimately the posterior showed limited coverage even after ten thousand sample burn in per chain. Below, I reproduce the (Smets and Wouters, 2007) using 10 million MCMC samples instead. Again only the first 12 parameters are produced to reduce clutter. The rest are displayed in the end of Section BE. of the Appendix.

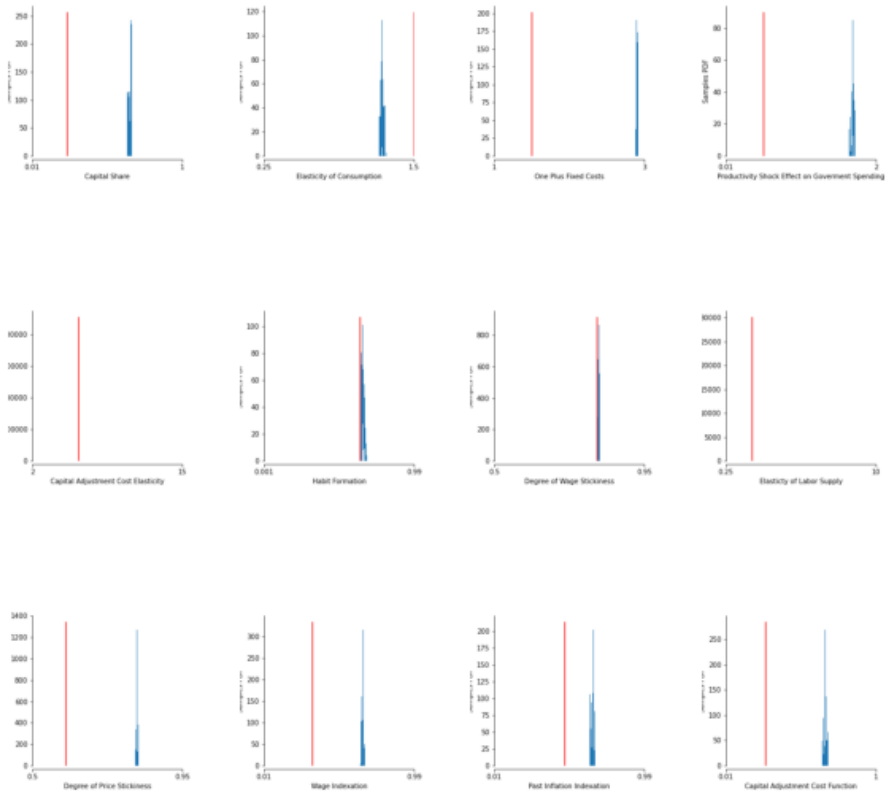

Figure IX: Smets-Wouters MH-MCMC Posterior with Parameters 1-12

Both algorithms, MCMC and SNPE, call the Smets and Wouters (2007) Dynare code when solving the model. The performance of the MCMC algorithm, as applied in this study, presents several challenges in terms of its ability to converge to the true data generating parameters, as can be seen in the chart. While some of the posteriors show good coverage of the true parameters, a substantial number of the posteriors exhibit a support outside of the data generating parameter

value and often converge to delta functions far from the true parameters. Among many, capital share and one plus fix costs seem to be problematic with the MCMC algorithm placing all its mass near .8, and 2.9 receptively whereas the true parameter is located at .24 and 1.5. On the other hand, the SNPE posterior presents a more reasonable representation, concentrating mass around the true parameter values. Although MCMC learns a posterior concentrated around the true value of habit formation, it repeats this error for most of the other 12 parameters including elasticity of labor supply, price stickiness, capital adjustment function among others. Additionally, the posteriors in SNPE have no issue reproducing multi-modal posteriors, a documented problem with MCMC, illustrated in Herbst and Schorfheide (2014). The other 24 parameters are similarly problematic for MCMC, while also making sense when estimated via SNPE.

In the cases of uninformative priors, there is evidence for the difficulty converging under realistic computational resources for MCMC. It is widely known that MCMC or even improvements like HMC empirically converge slower than other Machine Learning alternatives (Nathoo et al., 2013). Additionally, Herbst and Schorfheide (2014) demonstrate the identification problem for MCMC in the Smets-Wouters model, while Andrews and Mikusheva (2015) highlight theoretical problems in identifying DSGE models in the MLE setting. When using MCMC-based simulated annealing to perform MLE optimization, Siarry et al. (1997) demonstrate curse of dimensionalty issues even with objective functions with as few as 3-6 parameters. Additionally Chewi (2023) points out that for multi-modal and even unidentified distributions, MCMC and related approaches like HMC and No U-Turn Sampling (Hoffman, Gelman et al., 2014), converges at a rate slower than polynomial, while there is reason to suspect SNPE converges quadratically or perhaps slightly slower, but still at a polynomial rate (Farrell, Liang and Misra, 2018), (Draxler, Schnörr and Köthe, 2022).

The SNPE algorithm used in this study was implemented in Python which called the Smets-Wouters solution Matlab code, while the MCMC algorithm was fully executed in native Matlab calling the package, Dynare, in the same way the Smets-Wouters code is called from the authors with minor modifications. However, due to the difference in programming languages and the overhead of calling Matlab code in Python, the estimation process was much slower for the SNPE algorithm, with roughly 2 models solved per second, compared to about 100 models solved per second for

the MCMC. If written in Python, test indicated that MCMC would be significantly slower with roughly the same number of iterations per second as SNPE. While 10 million samples of MCMC in is faster than five hundred thousand Python iterations, at this point of MCMC other constraints are being met. For example RAM memory on my computer was starting to reach it's limit, both during estimation, and also when attempting to graph the marginal posteriors from samples. This suggest sampling significantly more, which estimating this model needs, will likely require more involved software engineering. Nevertheless, to avoid any suggestion of unfairness comparison, the SNPE posterior is shown below after 150 thousand samples (as opposed to 500 thousand samples in Figure VIII), which took roughly the same 30 hours as the 10 million samples of the MCMC. Again only the first 12 parameters are provided to reduce clutter:

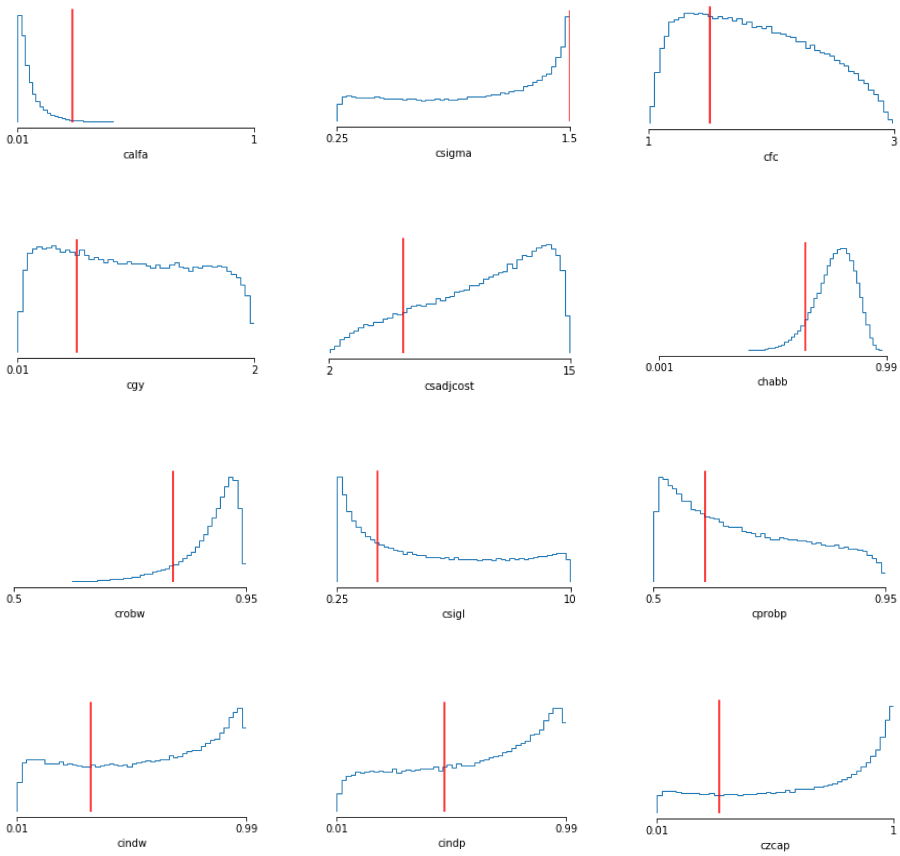

Figure X: Smets-Wouters SNPE Posterior with 150 Thousand Samples: Parameters 1-12

Although the SNPE algorithm incurs a Python time penalty, it still manages to learn a better posterior in the same amount of time as the MCMC written in Matlab, as discussed earlier. The performance of the SNPE algorithm can be further improved by optimizing the Python code through techniques such as Numba or writing the back-end algorithm in C++. However, these are beyond the scope of this paper. The results indicate that SNPE posterior is not perfect, but, based on the 500 thousand sample posterior, it still assigns a significant probability mass to the true data-generating process and in some cases, even concentrates around the true parameters. This is a marked improvement compared to the MCMC algorithm, which even after 10 million iterations, generally only produces highly concentrated posteriors along a small range that often does not

include the true data-generating parameter. Now I will move on to discuss heterogeneous agent models.

IV.G. HANK Model

The sixth model is the 10-parameter HANK model with the parameters discussed in Section BF. of the Appendix. Figure BF. is a chart of SNPE estimating a 10-parameter HANK model solved via Reiter:

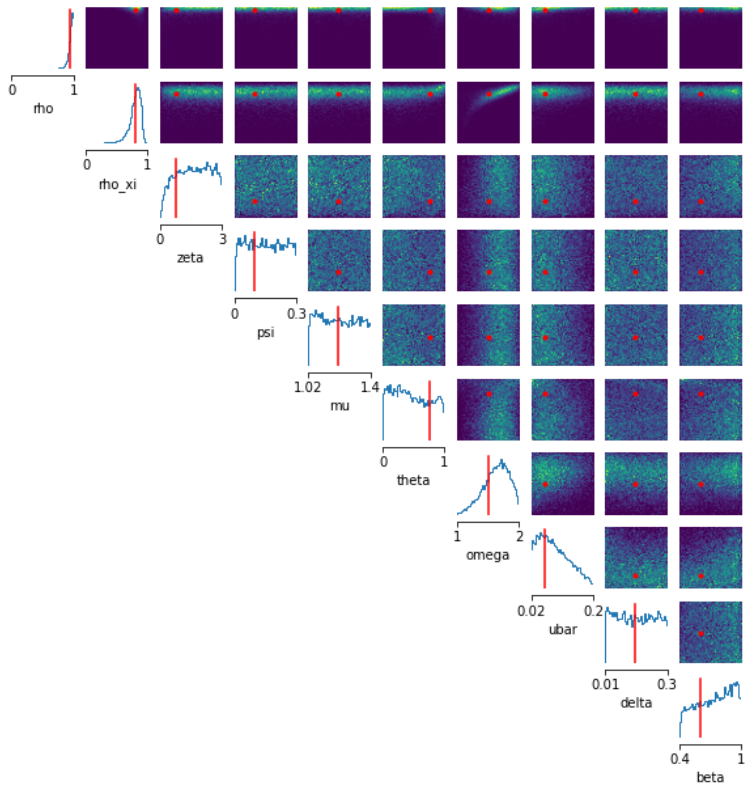

Figure XI: Ten Parameter HANK model–Partially Observed

The HANK model features two observable shocks, namely the unemployment rate and the interest rate, and two observable equations. It was solved with Reiter's method (Reiter, 2009). Despite having a large latent state-space of 810 latent variables, the estimation of the model was carried out successfully without the need for lossy dimensionality reduction of Ahn et al. (2018). Dimensionality

reduction is needed for estimating models solved via Reiter's method when evaluating the likelihood. This is because filtering becomes intractable as the size of the state-space grows. However, SNPE does not require estimating with a likelihood function and, by extension, the intermediate filtering step. As one can see, the approach used in the estimation process was able to identify the parameters with confidence, and the estimated posteriors agrees with the data generating process. This demonstrates the feasibility of estimating HANK models with a large latent state-space, via SNPE, which is computationally infeasible for likelihood approaches.

The algorithm was able to estimate the parameters ρ, ρ_ξ, ω and $\bar{u}$ with high accuracy. The mode of the distribution of these parameters was found to be either at or close to the true parameter value. However, for the other parameters, the likelihoods were found to be flat, which may be attributed to the limited impact of these parameters on the chosen observed variables or the relatively small number of time steps in my data (200).

An alternative Deep Learning approach for HANK estimation is Kase, Melosi and Rottner (2022). They use a Deep Learning approach to learn the relationship between the parameter values and the likelihood. Using this "extended" neural network, they can query from the likelihood using the network as a surrogate without having to solve the model. Both SNPE and the Kase, Melosi and Rottner (2022) approach use a surrogate model. However, in my case, I learn a posterior from samples rather than learning a likelihood from likelihood queries. Then once the likelihood is learned, they perform MCMC by using the surrogate likelihood providing likelihood information. It is unclear if predicting the likelihood is more efficient than density estimating from samples. Their application uses tight priors with modes precisely at the true parameter value. Thus, even if the likelihood was not informative, their approach would recover the true parameter values. At the same time my approach uses uniform priors which are diffuse, and still often recover the true data generating process at my posterior modes.

In conclusion, with SNPE, the estimation of the HANK model showed promising results, and highlights the potential of the approach used in the estimation process for models with large latent state-spaces.

IV.H. Winberry Heterogenous Agent Model

A seventh model is a Winberry (2018) solution method to the Krusell and Smith (1998) model with full micro-moments information as implemented in Liu and Plagborg-Møller (2023). SNPE recovers similar posterior results as Liu and Plagborg-Møller (2023), but is around 13x faster per iteration. This is because I only need micro and macro variable samples, and do not need to reconstruct the likelihood with an expensive sampling approach. I use the same number of iterations (10000) as Liu and Plagborg-Møller (2023), and this seems to converge to the same posterior distribution. I attempted to estimate even more parameters, which was not shown in Liu and Plagborg-Møller (2023). The estimation routine results in posteriors that are somewhat diffuse. However, since SNPE produces more reproducible posteriors, even when they are flat, the model can still be reproduced. Figure XII shows good identification on the smaller 3 parameter set:

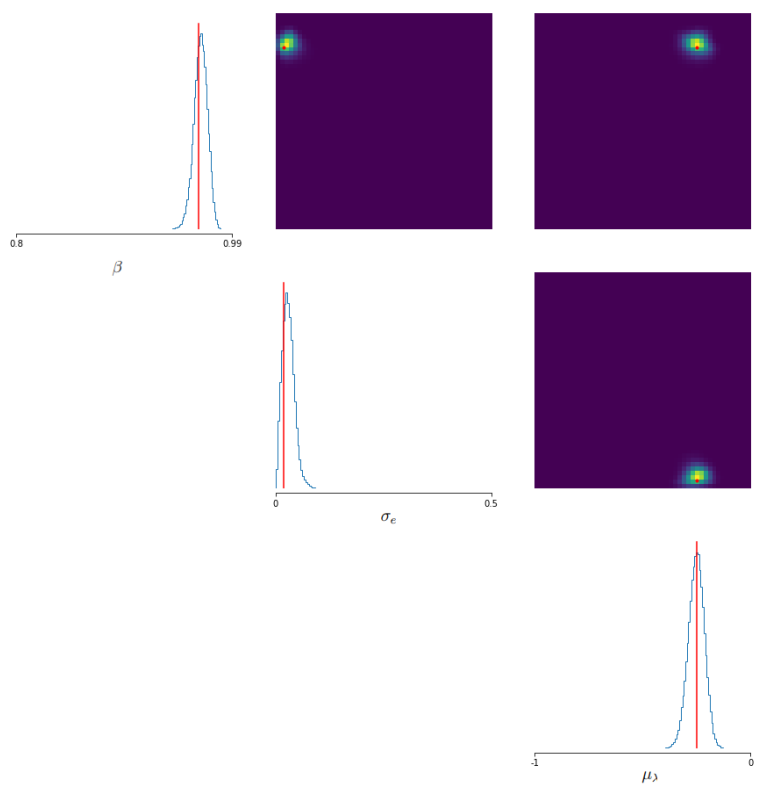

Figure XII: SNVI Results

As you can see the posterior concentrates around the true data generating parameters.

Figure XIII are the Liu and Plagborg-Møller (2023) results:

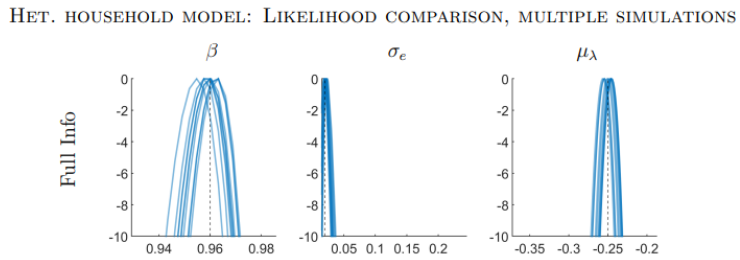

Figure XIII: Liu and Plagborg-Moller Results

Figure XII shows that the posterior concentrates almost entirely around the true parameter value, suggesting that the estimation was highly informative regarding the parameter value and accurately reproduces the simulation parameters. The results are compared to those obtained by Liu and Plagborg-Møller (2023) (Figure XIII). Both sets of results are similar and demonstrate the viability of the approach used in this study. This example illustrates how SNPE is a fast and reliable approach to the estimation of heterogeneous agent macroeconomic models.

IV.I. Empirical Application

Additionally, using the Winberry (2018) solution method, and extending Liu and Plagborg-Møller (2023), I estimated Krusell and Smith (1998) model on empirical data using both micro wealth percentiles, and macro variables: output, investment, consumption and wages. The posterior variables estimated are β, the discount rate; μ_l, the labor productivity heterogeneity parameter; measurement error for each of the 4 macro variables and the micro wealth process (meas_error 5); and a lower bound on net borrowing.

The macro data comes from standard sources (FRED, etc.). The micro data comes from the total household net worth variable in the Survey of Income and Program Participation (SIPP). One problem with the SIPP data is the number of people interviewed in the years with interviews, ranges from 700 people to 1700 people. A common way to deal with this is to match 100 samples

with a 100 random sample from the data. This throws away a information. Due to the simulation advantages of SNPE, I can match percentiles transforming the empirical data into the first to 99th percentiles, and matching that with model generated percentiles. This provides more information and implicitly use the entire micro data set to discipline the model. If one wanted to match percentiles with a likelihood approach, one would have to calculate 99 order statistics, which is unmanageable. However, the downside of this approach is that one can no longer determine the joint relationship of wealth and employment. One can still sum employment across agents to form a macro unemployment rate. Figure XIV below shows the results of the empirical exercise:

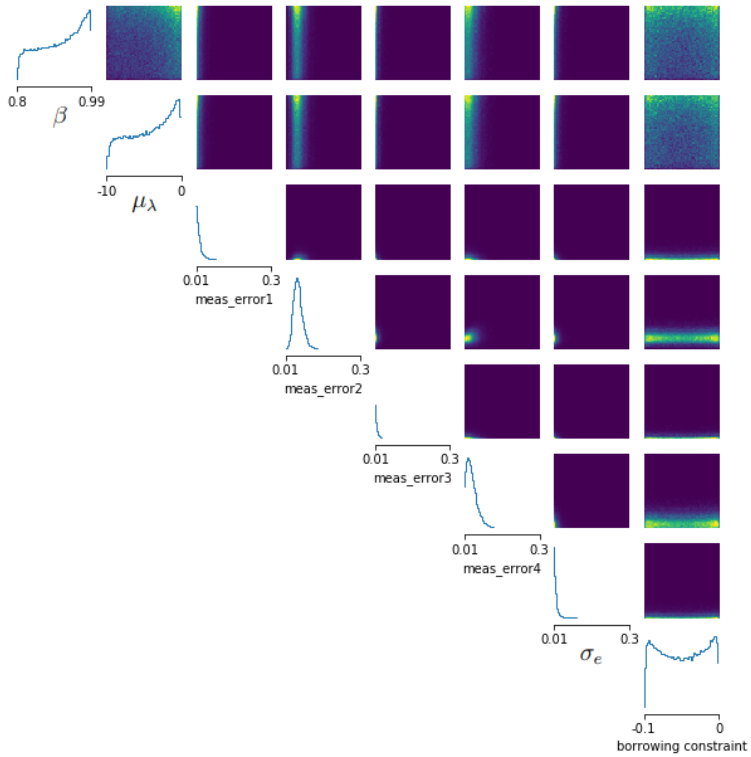

Figure XIV: Krussel Smith/Winberry Empirical Results

The parameters of interest are all somewhat diffuse, although the modes make sense. The discount rate is close to .99. The second term, the labor productivity parameters (μ_λ), with a mode around -.25, implies a standard deviation (heterogeneity) of the labor productivity process

that Liu and Plagborg-Møller (2023) point out is in line with Piketty, Saez and Zucman (2018). The empirical exercise suggests this model has decent, but not exceptional, identification of the heterogeneous parameter of interest, μ_λ.

V. Conclusion

In this paper, I introduce a novel approach, SNPE, to estimate models in a Bayesian and simulation-based framework. SNPE does not require a likelihood function and also derives posteriors faster and more accurately than baseline MCMC and other competing algorithms. The model can Bayesian estimate relatively large models like the 36 parameter Smets and Wouters (2007) model. The algorithm has also been applied to HANK models with large latent state-spaces and has been successful in providing credible posterior estimates faster than baseline algorithms. Moreover, it can be applied to solution methods such as value function iteration and projection, which do not yield a likelihood function. This approach can be applied as broadly as MSM with improved efficiency and the ability to posterior estimate. The algorithm can take in any solution method and estimate any black box model as long as the model can produce simulated data.

The proposed approach has a wider range of applications beyond macroeconomics. It can be used in the fields of industrial organization, applied game theory, and econometrics. In industrial organization and game theory, the algorithm can be used to solve structural and dynamic models. In econometrics, it can be applied to estimate general latent time series models effectively and efficiently, without the need for a likelihood function. Additionally, the the suite of conditional density estimators can be used for density estimation in high-dimensional settings.

In conclusion, SNPE can estimate more models faster and more accurately out of the box without modifying the solution method. Additionally it has a much broader scope of application than just macroeconomic modeling. The applications of this technique are not limited to this paper, and have the potential to be explored in greater depth in future research.

References

Adjemian, Stéphane, Houtan Bastani, Michel Juillard, Fréderic Karamé, Ferhat Mihoubi, Willi Mutschler, Johannes Pfeifer, Marco Ratto, Normann Rion, and Sébastien Villemot. 2022. "Dynare: Reference Manual Version 5." CEPREMAP Dynare Working Papers 72.

Ahn, SeHyoun, Greg Kaplan, Benjamin Moll, Thomas Winberry, and Christian Wolf. 2018. "When inequality matters for macro and macro matters for inequality." *NBER macroeconomics annual*, 32(1): 1–75.

Andrews, Isaiah, and Anna Mikusheva. 2015. "Maximum likelihood inference in weakly identified dynamic stochastic general equilibrium models." *Quantitative Economics*, 6(1): 123–152.

Auclert, Adrien, Bence Bardóczy, Matthew Rognlie, and Ludwig Straub. 2021*a*. "Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models." *Econometrica*, 89(5): 2375–2408.

Auclert, Adrien, Hannes Malmberg, Frederic Martenet, and Matthew Rognlie. 2021*b*. "Demographics, wealth, and global imbalances in the twenty-first century." National Bureau of Economic Research.

Barron, Andrew R. 1993. "Universal approximation bounds for superpositions of a sigmoidal function." *IEEE Transactions on Information theory*, 39(3): 930–945.

Bishop, Christopher M. 1994. "Mixture density networks."

Caballero, Ricardo J, and Eduardo MRA Engel. 2007. "Price stickiness in Ss models: New interpretations of old results." *Journal of monetary economics*, 54: 100–121.

Caglio, Cecilia R, R Matthew Darst, and Sebnem Kalemli-Özcan. 2021. "Risk-taking and monetary policy transmission: Evidence from loans to smes and large firms." National Bureau of Economic Research.

Chewi, Sinho. 2023. "Log Concave Sampling."

Childers, David, Jesús Fernández-Villaverde, Jesse Perla, Christopher Rackauckas, and Peifan Wu. 2022. "Differentiable State-Space Models and Hamiltonian Monte Carlo Estimation." National Bureau of Economic Research.

Dixit, Avinash K, and Joseph E Stiglitz. 1977. "Monopolistic competition and optimum product diversity." *The American economic review*, 67(3): 297–308.

Draxler, Felix, Christoph Schnörr, and Ullrich Köthe. 2022. "Whitening Convergence Rate of Coupling-based Normalizing Flows." *Advances in Neural Information Processing Systems*, 35: 37241–37253.

Duffie, Darrell, and Kenneth J Singleton. 1990. "Simulated moments estimation of Markov models of asset prices."

Durkan, Conor, Iain Murray, and George Papamakarios. 2020. "On contrastive learning for likelihood-free inference." 2771–2781, PMLR.

Farrell, Max H, Tengyuan Liang, and Sanjog Misra. 2018. "Deep neural networks for estimation and inference: Application to causal effects and other semiparametric estimands." *arXiv preprint arXiv:1809.09953*.

Gallant, A Ronald, and George Tauchen. 1996. "Which moments to match?" *Econometric theory*, 12(4): 657–681.

Glöckler, Manuel, Michael Deistler, and Jakob H Macke. n.d.. "Variational methods for simulation-based inference."

Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. "Generative adversarial nets." *Advances in neural information processing systems*, 27.

Gourieroux, Christian, Alain Monfort, and Eric Renault. 1993. "Indirect inference." *Journal of applied econometrics*, 8(S1): S85–S118.

Greenberg, David, Marcel Nonnenmacher, and Jakob Macke. 2019. "Automatic posterior transformation for likelihood-free inference." 2404–2414, PMLR.

Haan, Peter, and Arne Uhlendorff. 2006. "Estimation of multinomial logit models with unobserved heterogeneity using maximum simulated likelihood." *The Stata Journal*, 6(2): 229–245.

Hajivassiliou, Vassilis A, et al. 1997. *Some practical issues in maximum simulated likelihood.* Suntory-Toyota International Centre for Economics and Related Disciplines ....

Hennessy, Christopher A, and Toni M Whited. 2007. "How costly is external financing? Evidence from a structural estimation." *The Journal of Finance*, 62(4): 1705–1745.

Herbst, Edward, and Frank Schorfheide. 2014. "Sequential Monte Carlo sampling for DSGE models." *Journal of Applied Econometrics*, 29(7): 1073–1098.

Herbst, Edward P, and Frank Schorfheide. 2015. *Bayesian estimation of DSGE models.* Princeton University Press.

Hermans, Joeri, Volodimir Begy, and Gilles Louppe. 2020. "Likelihood-free mcmc with amortized approximate ratio estimators." 4239–4248, PMLR.

Hoffman, Matthew D, Andrew Gelman, et al. 2014. "The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo." *J. Mach. Learn. Res.*, 15(1): 1593–1623.

Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer feedforward networks are universal approximators." *Neural networks*, 2(5): 359–366.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Kaji, Tetsuya, Elena Manresa, and Guillaume Pouliot. 2020. "An adversarial approach to structural estimation." *arXiv preprint arXiv:2007.06169.*

Kase, Hanno, Leonardo Melosi, and Matthias Rottner. 2022. "Estimating nonlinear heterogeneous agents models with neural networks."

Keane, Michael P, and Kenneth I Wolpin. 1994. "The solution and estimation of discrete choice dynamic programming models by simulation and interpolation: Monte Carlo evidence." *the Review of economics and statistics*, 648–672.

Khan, Aubhik, and Julia K Thomas. 2008. "Idiosyncratic shocks and the role of nonconvexities in plant and aggregate investment dynamics." *Econometrica*, 76(2): 395–436.

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

Krusell, Per, and Anthony A Smith, Jr. 1998. "Income and wealth heterogeneity in the macroeconomy." *Journal of political Economy*, 106(5): 867–896.

Lerman, Steven, and Charles Manski. 1981. "On the use of simulated frequencies to approximate choice probabilities." *Structural analysis of discrete data with econometric applications*, 10: 305–319.

Liu, Laura, and Mikkel Plagborg-Møller. 2023. "Full-information estimation of heterogeneous agent models using macro and micro data." *Quantitative Economics*, 14(1): 1–35.

Loaiza-Maya, Rubén, Michael Stanley Smith, David J Nott, and Peter J Danaher. 2022. "Fast and accurate variational inference for models with many latent variables." *Journal of Econometrics*, 230(2): 339–362.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Mohammad-Djafari, Ali, and Hacheme Ayasso. 2009. "Variational Bayes and mean field approximations for Markov field unsupervised estimation." 1–6, IEEE.

Nathoo, FS, ML Lesperance, AB Lawson, and CB Dean. 2013. "Comparing variational Bayes with Markov chain Monte Carlo for Bayesian computation in neuroimaging." *Statistical methods in medical research*, 22(4): 398–423.

Ottonello, Pablo, and Thomas Winberry. 2020. "Financial heterogeneity and the investment channel of monetary policy." *Econometrica*, 88(6): 2473–2502.

Pakes, Ariel, and David Pollard. 1989. "Simulation and the asymptotics of optimization estimators." *Econometrica: Journal of the Econometric Society*, 1027–1057.

Papamakarios, George, and Iain Murray. 2016. "Fast ε-free inference of simulation models with bayesian conditional density estimation." *Advances in neural information processing systems*, 29.

Piketty, Thomas, Emmanuel Saez, and Gabriel Zucman. 2018. "Vol. 133 May 2018 Issue 2." *The Quarterly Journal of Economics*, 553: 609.

Reiter, Michael. 2009. "Solving heterogeneous-agent models by projection and perturbation." *Journal of Economic Dynamics and Control*, 33(3): 649–665.

Rendahl, Pontus. 2017. "Linear time iteration." IHS Economics Series.

Rubin, Donald B. 1984. "Bayesianly justifiable and relevant frequency calculations for the applied statistician." *The Annals of Statistics*, 1151–1172.

Schmitt-Grohé, Stephanie, and Martın Uribe. 2004. "Solving dynamic general equilibrium models using a second-order approximation to the policy function." *Journal of economic dynamics and control*, 28(4): 755–775.

Siarry, Patrick, Gérard Berthiau, François Durdin, and Jacques Haussy. 1997. "Enhanced simulated annealing for globally minimizing functions of many-continuous variables." *ACM Transactions on Mathematical Software (TOMS)*, 23(2): 209–228.

Smets, Frank, and Rafael Wouters. 2007. "Shocks and frictions in US business cycles: A Bayesian DSGE approach." *American economic review*, 97(3): 586–606.

Wainwright, Martin J, and Michael Irwin Jordan. 2008. *Graphical models, exponential families, and variational inference.* Now Publishers Inc.

Winberry, Thomas. 2018. "A method for solving and estimating heterogeneous agent macro models." *Quantitative Economics*, 9(3): 1123–1151.

Appendix

A Bayesian Estimation in General and MCMC

The predominant technique for performing Bayesian inference in economics is MCMC (Herbst and Schorfheide, 2015). In MCMC, one only concerns oneself with relative probablity jumps between parameter values. For example, assume one knows both the prior and the likelihood for any given point, $P(X|\theta)P(\theta)$, then one knows the relative likelihood of being at a point, θ versus any other, θ': $\frac{P(X|\theta)P(\theta)}{P(X|\theta')P(\theta')}$. One can calculate this relative likelihood of sampling any two points in the posterior, even though the constant normalization term in Bayes rule, $P(x)$ is not known. Since one knows the relative probabilities, one can design a random walk so that a walker visits θ points in proportion to their relative probabilities. One way to do that is if a walker can choose to compare any point in the posterior to the point the walker is currently at, the walker will move to the second point with the likelihood according to the ratio of the probabilities. For example, if the second point is half as likely to be in the posterior, the walker will move with probability one half and probability one half generate another sample from the same intial point. If the other point is, for example, 50 percent more likely: $\frac{1.5}{1}$, the walker will move to the second point for certain, with the understanding that if the walker randomly selected to move back, the move back would be with $\frac{1}{1.5}$ probability. Figure XV, shows the random walk algorithm exploring a Gaussian distribution including some samples that were rejected due to low probability of occurring.

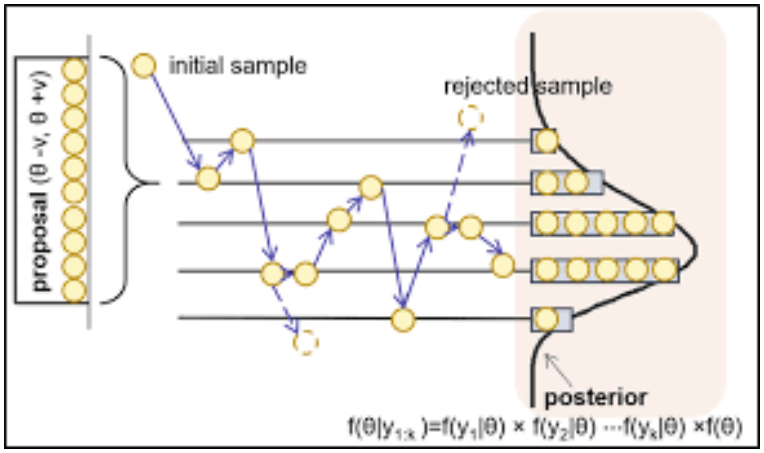

Figure XV: Random Walk MCMC

In this way, the walker moves across the space of the posterior with relative frequency according to $P(X|\theta)P(\theta)$ ratio between points, which ultimately means that one is sampling from $P(\theta|X)$ without calculating the normalizing partition function. In practice, MCMC is a little more complicated, as often the parameter space is unbounded, and one can't sample the entire space with equal probability. Then this involves the use of a proposal distribution and the use of importance sampling, where one samples from closer points with higher probability, but the likelihood is normalized to take into account the fact that nearer points are more likely to be selected than further points. This allows MCMC to sample in the long run from the entire space as long as one adjusts for the probability that the proposal distribution samples unequally from the posterior space. In particular, call $P(X|\theta)P(\theta)$ the unnormalized posterior for a point θ, and $P(X|\theta')P(\theta')$ the unnormalized posterior for a point θ'. Given a proposal distribution, $f(\theta)$, which is the probability of visiting θ from θ', and $f(\theta')$ as the probability of visiting θ' from θ, then the equation that preserves the relative ratios and samples from the posterior will move to the new point with probability:

$$\frac{\frac{P(X|\theta)P(\theta)}{f(\theta)}}{\frac{P(X|\theta')P(\theta')}{f(\theta')}}$$

Since often the f distribution is assumed to be symmetric, the f terms drop out leaving the ratios

of the unnormalized probabilities used to decide when to walk. Unsurprisingly, this approaches is quite slow as one has to visit the whole space sequentially with higher probability points being visited many more times than lower probability points. Alternatives like variational inference have been proposed which speed up the inference at the cost of some bias (Wainwright and Jordan, 2008), but it is beyond the scope of this paper. The approach I propose, SNPE, does not require knowing the likelihood, something that may be intractable to calculate, depending on the solution algorithm for macroeconomic models.

AA. Feed-Forward Neural Networks

At its core, Feed Forward Neural Networks are compositions of vector-valued linear equations with non-linearities between each composition. For example, a neural network with a single hidden layer could be constructed:

$$Y = W_2 * \sigma(W_1 * X + b_1) + b_2 \tag{9}$$

Here X is some vector-valued input with dimensionality n by assumption. W_1 is a k by n matrix that turns an n-dimensional input into a k-dimensional basis for the second layer. Likewise b_1 is a k-dimensional intercept term. W_2 turns the k-dimensional basis vector from the first linear regression into the m dimensional output vector and b_2 is the m-dimensional intercept term. Since W_1, W_2, b_1, b_2, are all linear operations, arbitrary compositions of them will still be linear. However if you add a non-linearity term, σ between the compositions, with loose regularity conditions, a single hidden layer neural network as shown above can approximate any continuous function (Hornik, Stinchcombe and White, 1989), as the k-dimensional basis expands to infinity. The non-linearity can be almost any function. The one I use is the Leaky ReLU, which is shown as the σ function applied to the first vector equation, $W_1 * X + b_1$, below at Figure XVI:

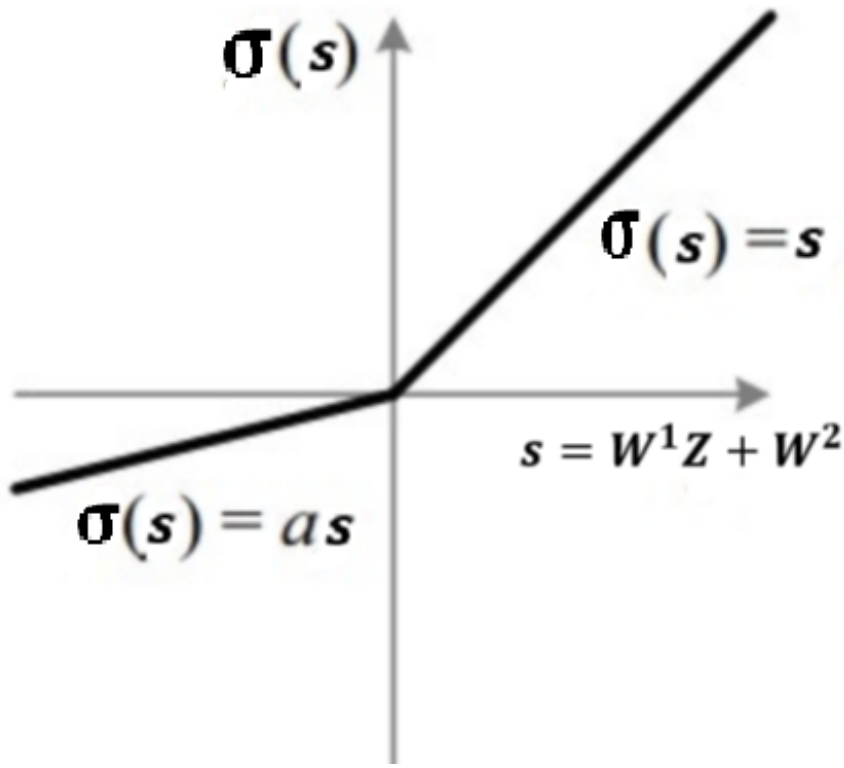

Figure XVI: Feed Forward Neural Network

This transforms the output of $W_1 * X + b_1$ the X axis, transformed by the kinked nonlinearity, to produce the Y axis output, in order to make the neural network more expressive. Other non-linearities include Logits, hyperbolic tangents, ReLUs, and ELUs which are modifications of the Leaky ReLU. Additionally, one can make deeper neural networks by composing more vector linear regressions with non-linearities, parameterized by, W_3, and b_3, for example.

Here is a graphical representation of layers in a neural network stacked one on top of another. Usually Feed Forward Neural Networks have more than one hidden layer:

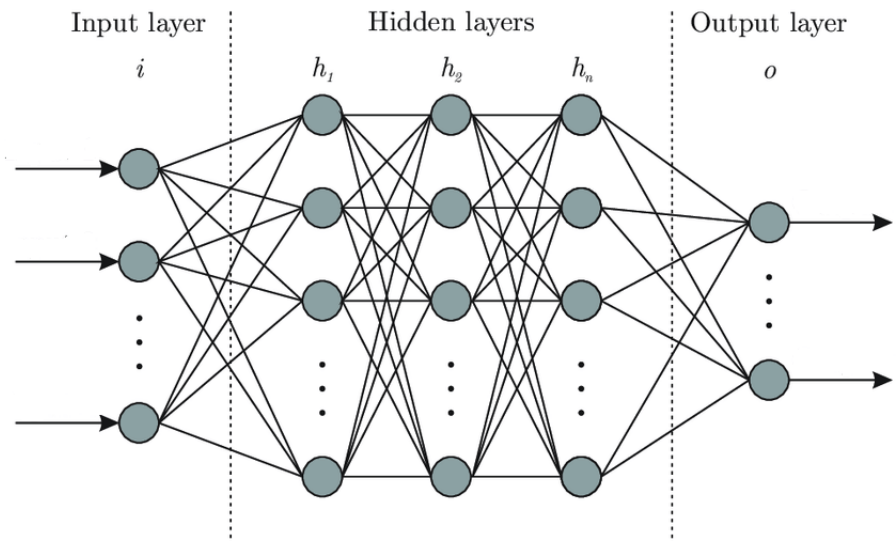

Figure XVII: Feed Forward Neural Network

In this case, there are three hidden layer in combination with the input vector and the output vector. These layers can be arbitrarily wide, with width and depth associated with increased flexibility but also increased parameter count for risk of over-fitting.

B Economic Models

I apply SNPE and other baseline estimation algorithms on a suite of economic models. This section will discuss the economic details of the models being estimated.

BA. Real Business Cycle Model

The results of estimating the REal Business Cycle (RBC) model using the SNPE algorithm are presented and discussed. The RBC Model is standard in macroeconomics. This version has four parameters – α, β, δ, and ρ – with a fixed Constant Relative Risk Adverssion (CRRA) elasticity of 2. The first 100 iterations are dropped to attain a steady-state behavior. This RBC model comes from Alisdair McKay's note on heterogeneous agent models: `https://alisdairmckay.com/Notes/HetAgents/index.html`.

The utility function is a standard CRRA utility:

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{10}$$

The production function is standard Cobb-Douglas with productivity:

$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{11}$$

Where $\bar{L}$ is fixed labor and Z_t is productivity and K_t is capital which evolves according to a standard law of motion:

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{12}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{13}$$

This model is solved via time iteration (Rendahl, 2017) and matches consumption, investment, and productivity to real data. Interest rates and capital are the latent unobserved variables. This is done on data both synthetic and real.

The results of the posterior are displayed below in Figure XVIII, where the red bar represents the true parameter values and the blue line represents the posterior as estimated by the simulation-based inference approach:

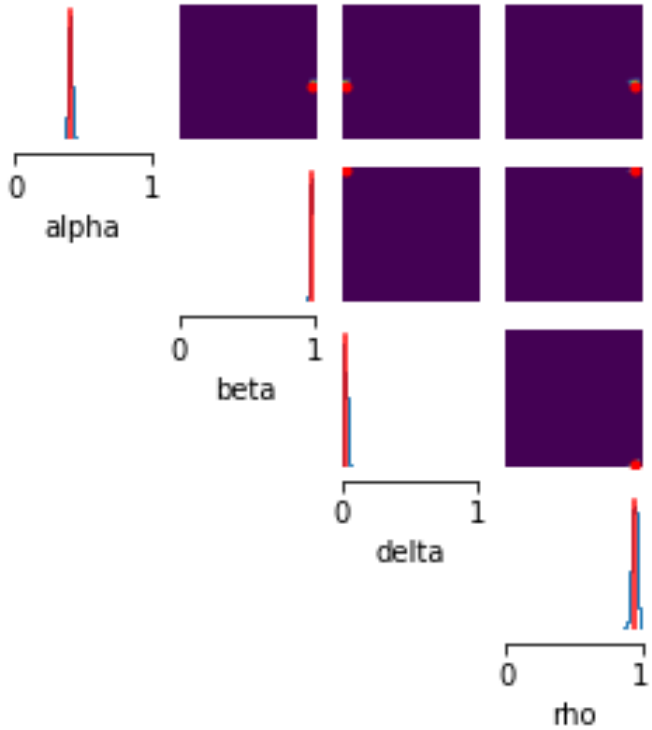

Figure XVIII: Simulated Data RBC

The main diagonal density plots in the figure shows the posterior marginals. The off-diagonal heat maps show two-way densities, with the color indicating the probability mass. As you can see, the posterior concentrates on the true data generating parameter.

The approximate true posterior is obtained by running the MCMC for 2 million iterations as show below, to approximate the ground truth:

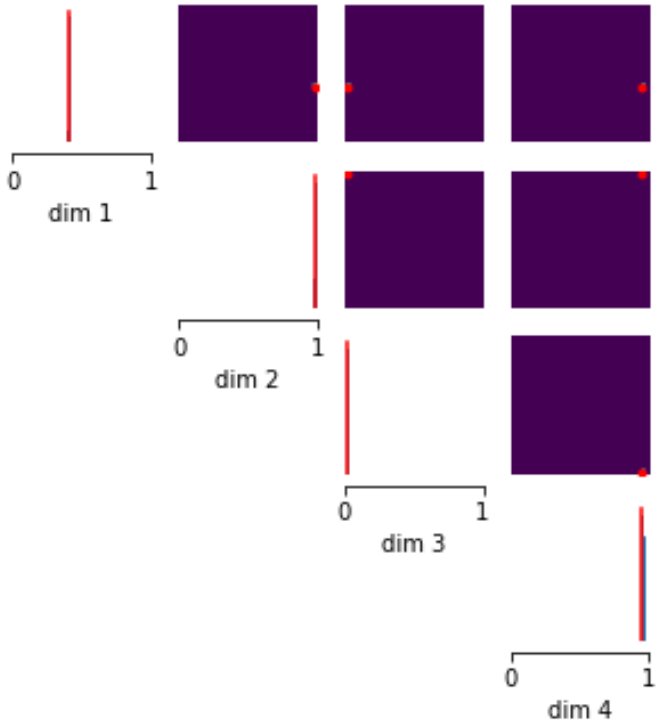

Figure XIX: Simulated Data RBC

SNPE converges to the MCMC ground truth solution, with significantly less iterations.

Figure XX below shows the same RBC model is also estimated on real data, where there is no available ground truth. The parameters derived from the real data are not necessarily realistic, which is expected as the RBC model is a simple model. The estimation approach is uncertain about identification, as indicated by the spread of the posterior. The model also predicts some parameters that are not in line with theory, such as the discount rate, β, and the parameter on the productivity process, ρ. The depreciation rate, δ, is the only parameter that is relatively reasonable. Below is the posterior estimated on real data:

In conclusion, the results indicate that the SNPE algorithm can estimate the RBC model to a good degree of accuracy. The RBC model's limitations in real-world applications are also highlighted, as it fails to estimate some parameters that are not in line with theoretical predictions.

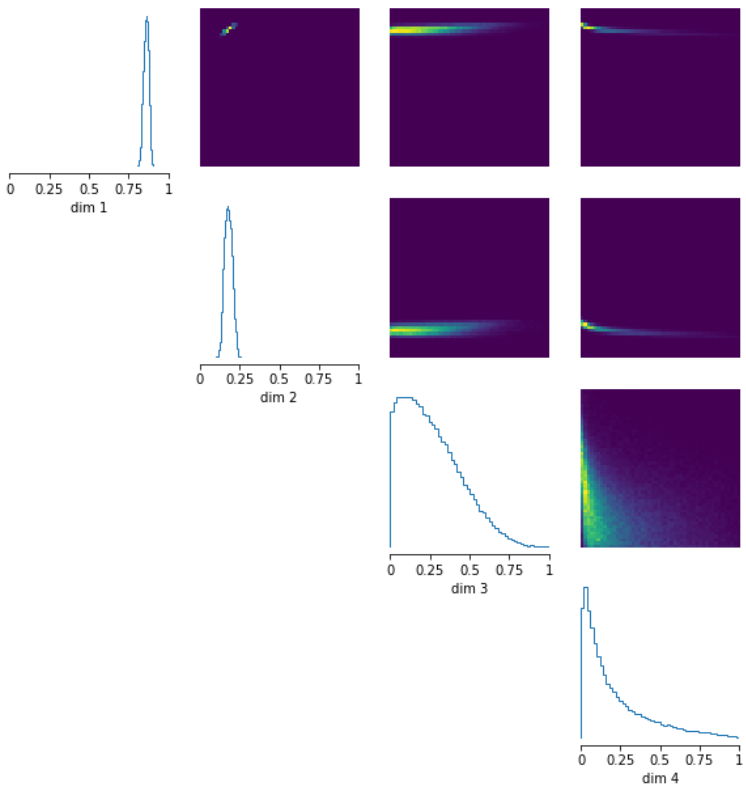

Figure XX: Real Data RBC

This highlights the need for more complex models to better reflect real-world economic scenarios, which will be illustrated in the next models.

BB. Cash Flow Model

In this section, I am estimating a partial equilibrium model of firm cash flow by utilizing value function iteration. While this is not quite a macro model, as a structural approach the techniques used are the same. The model is specified with four parameters – α, δ, σ, and ρ. α is the capital share in the production function. δ is depreciation rate of capital. σ is the standard deviation of log productivity AR process. ρ is the slope of the log productivity AR process. The interest rate was set at 5 percent and the discount factor was calculated as 1/(1+r).

This model is a straightforward infinite horizon partial equilibrium model with risk neutral managers choosing investment to maximize the present value of a stream of firm cash flows and comes from solution code written by Emil Lakkis and Toni Whited.

Utility/cash flows are give by this formula:

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{14}$$

Where K_t, Z_t, and I_t are the same as in the RBC model – capital, productivity and investment respectively. π is the profit function.

The capital law of motion is like the RBC model:

$$K_t = (1 - \delta)K_{t-1} + I_t \tag{15}$$

Productivity evolves according to a AR(1) process in logs:

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{16}$$

Profit is defined as:

$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{17}$$

This model shares similarities with the RBC model, with the main difference being the utility function and the value function solution method. This estimation approach matches capital levels generated by a parameterized model. To estimate the model parameters, two different algorithms were applied – a simulation-based inference approach using SNPE, and SNVI. SNVI is discussed further in the Section CD. of the Appendix. The results are shown Figure XXI (SNPE) and Figure XXII (SNVI):

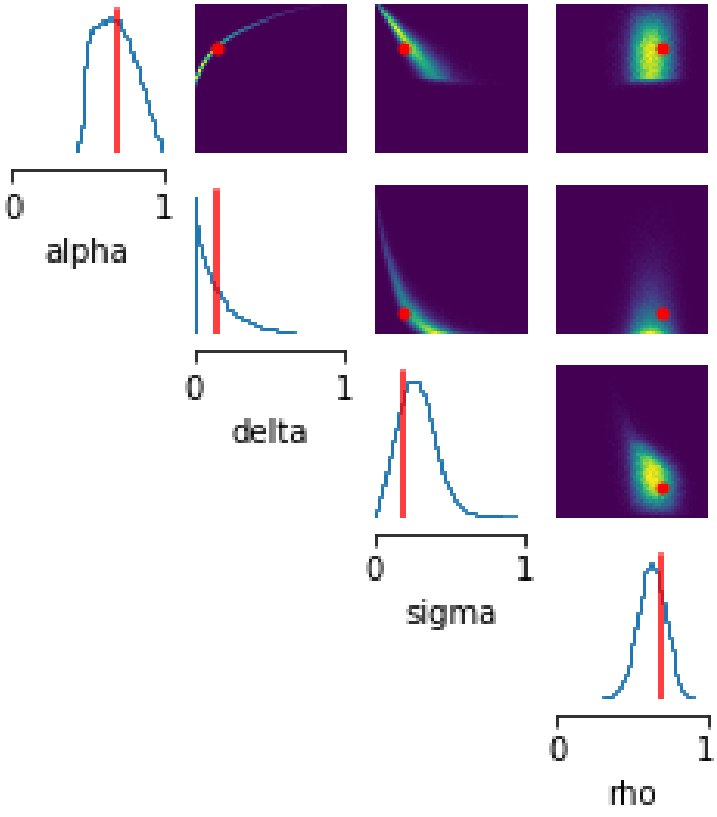

Figure XXI: Value Function Iteration Estimated with SNPE

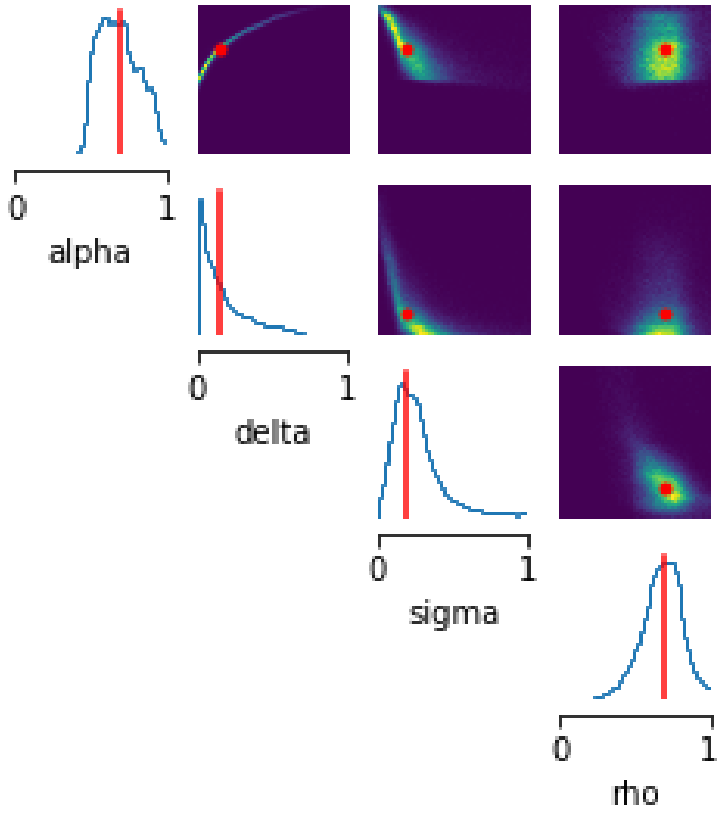

Figure XXII: Value Function Iteration Estimated with SNVI

From the visual inspection of the charts XXI and XXII, it can be observed that the posteriors obtained from the two estimation approaches are similar, suggesting that the same distribution was learned. Additionally, the mode of the distribution appears to be close to the actual parameter values, which indicates that the model is capable of accurately recovering the true parameters.

BC. Lucas Asset Pricing Projection Model

In the following section, we demonstrate the estimation procedure on a Lucas Asset Pricing Model, which is solved via projection without measurement error. Following Mohammed Ait Lahren's code (`https://notes.quantecon.org/submission/5f44808ad24fdb001162a53b`), each

agent maximizes CRRA utility:

$$max_i E_t[\beta^t \frac{c_t^{1-\gamma} - 1}{1-\gamma} \tag{18}$$

with γ set to 3. The law of motion is:

$$k_{t+1} = (1 + d_t/P_t)k_t - c_t/P_t \tag{19}$$

The state of the system is defined as dividends received plus price of the amount of assets owned:

$$m_{t+1} = (P_{t+1} + d_{t+1})k_{t+1} \tag{20}$$

Putting these three equations together one gets the pricing kernel:

$$p_t = \beta^t E_t[(\frac{c_{t+1}}{c_t})^{-\gamma}(d_{t+1} + p_{t+1})] \tag{21}$$

Dividends evolve under an $AR(1)$ process:

$$d_{t+1} = \mu_d + \rho_d * d_t + \epsilon \tag{22}$$

where ϵ is a shock with standard deviation σ. When behaving optimally, consumption also equals the dividends received. This model matches the expected return and risk free rate at a variety of dividend levels, which demonstrates the ability of this model to not only estimate on models solved via projection, but also to work on cross sectional problems as well.

The purpose of this estimation is to showcase the capability of the SNPE approach to estimate models solved via projection. The results are show in Figure XXIII:

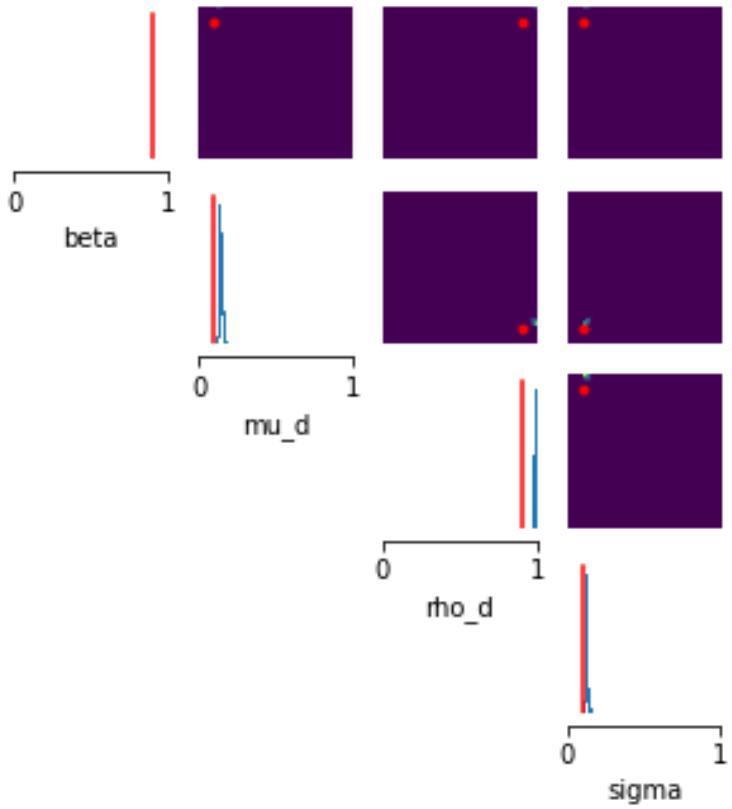

Figure XXIII: Projection Model

This estimation procedure took under 10 hours to complete on an 8-core Intel i7 machine, highlighting the computational efficiency of this approach. This result also demonstrates that models solved via projection can be effectively estimated without measurement error using the SNPE framework. The posterior accuracy is pretty good, but it seems like the posteriors have a mode that is slightly shifted and while close to the true parameter value, do not put much support on the true value along the μ_d and ρ_d dimensions.

BD. The Value Function Iteration Bequests Model

I estimate a model of bequests over time. The model illustrates the benefit of my approach as it cannot be estimated by perturbation because of the kink at the intersection of the consump-

tion versus adjusting illiquid savings intersection. Thus one should use simulation approaches to Bayesian estimate this model which is solved via value function iteration.

This model comes from Jeppe Druedahl's open source model example: `https://github.com/pkofod/vfi/blob/master/Fast%20VFI.ipynb`. The utility function is:

$$u(b_t, c_t) = \frac{[\phi(b_t + \underline{b})^{1-\gamma} + (1-\phi)c_t^{1-\gamma}]^{\frac{1-\rho}{1-\gamma}}}{1-\rho} \tag{23}$$

Here b_t is bequest or non-liquid savings. c_t is consumption. b, ϕ, γ, ρ are all parameters. The bequest utility function (end of life utility) is:

$$\nu(a_t, b_t) = \varphi \frac{(a_t + b_t + \underline{q})^{1-\vartheta}}{1-\vartheta} \tag{24}$$

Here a_t is liquid savings and q, θ, φ are all parameters. The value function is give by:

$$v_t(m_t, n_t, l_t) = \max\{v_t^{keep}(m_t, n_t, l_t), v_t^{adj}(m_t, n_t, l_t)\} \tag{25}$$

$$\text{s.t.} \tag{26}$$

$$x_t = m_t + (1-\tau)n_t \tag{27}$$

Here m_t is beginning of period liquid savings (analogous to end of period a_t) and n_t is beginning of period illiquid savings (analogous to end of period b_t). x_t is the pooled value of total liquid and

illiquid savings. l_t is labor income. The post-decision value function is:

$$w_t(a_t, b_t, l_t) = \nu(a_t, b_t), t = T \tag{28}$$

$$w_t(a_t, b_t, l_t) = \mathbb{E}_t \left[\max\{v_t^{keep}(m_{t+1}, n_{t+1}, l_{t+1}), v_t^{adj}(x_{t+1}, l_{t+1})\} \right], t < T \tag{29}$$

$$\text{s.t.} \tag{30}$$

$$l_{t+1} \sim F(l_t) \tag{31}$$

$$m_{t+1} = (1 + r_a)a_t + \omega l_{t+1} \tag{32}$$

$$n_{t+1} = (1 - \delta)b_t \tag{33}$$

$$x_{t+1} = m_{t+1} + (1 - \tau)n_{t+1} \tag{34}$$

Here labor income comes from a distribution $F(l_t)$. Likewise r_a, ω, δ, τ are all parameters. The keep value function is:

$$v_t^{keep}(m_t, n_t, l_t) = \max_{c_t \in [0, m_t]} u(n_t, c_t) + \beta w_t(a_t, b_t, l_t) \tag{35}$$

$$\text{s.t.} \tag{36}$$

$$a_t = m_t - c_t \tag{37}$$

$$b_t = n_t \tag{38}$$

The Keep Value Function is the value function at a point t if the agent chooses to consume and not withdraw or deposit funds into the illiquid asset, b_t. The adjust value function is:

$$v_t^{adj}(x_t, l_t) = \max_{b_t \in [0, x_t]} v_t^{keep}(m_t, n_t, l_t) \tag{39}$$

$$\text{s.t.} \tag{40}$$

$$m_t = x_t - b_t \tag{41}$$

$$n_t = b_t \tag{42}$$

The Adjust Value Function is the value of adjusting the illiquid asset. Notice the agent is unable

to consume this period. This cannot be effectively modeled with a approach like perturbation, because of its nonlinear and kinked decision rule. Even so, I use my estimation technique to get a Bayesian posterior distribution. This model matches the liquid asset, illiquid asset and consumption to synthetic data generated by a model. The chart is Figure VII in Section IV.E..

BE. Smets-Wouters 2007 Model

This section will mainly illustrate the equations and the parameters so one can cross reference parameters shown in the diagram with the correct equation in the DSGE model. This model comes from the paper Smets and Wouters (2007). More details on the economics of the equations can be found there. This model can be estimated both with MCMC and SNPE. Thus this model mostly illustrates the increases in posterior accuracy when using an uninformative prior which goes hand in hand with estimation speed when using SNPE vs MH-MCMC. Here is a reference for all the Smets-Wouter abbreviations and the actual variable name, written in order of appearance in the **?** picture of marginal posteriors distributions contained in Section IV.F. in the results.

calfa = Capital Share

csigma = Elasticity of Consumption

cfc = One Plus Fixed Costs

cgy = 'Productivity Shock Effect on Government Spending

csadjcost = Capital Adjustment Cost Elasticity

chabb = Habit Formation

cprobw = Degree of Wage Stickiness

csigl = Elasticity of Labor Supply

cprobp = Degree of Price Stickiness

cindw = Wage Indexation cindp = Past Inflation Indexation

czcap = Capital Adjustment Cost Function

crpi = Taylor Rule Inflation Coefficient

crr = Taylor Rule Degree of Interest Rate Smoothing

cry = Taylor Rule Output Gap Coefficient

crdy = Taylor Rule Output Growth Gap Coefficient

crhoa = AR Term on Productivity Shocks

crhob = AR Term on Risk Premium Shocks

crhog = 'AR Term on Government Spending Shocks

crhoqs = .AR Term in Investment Shocks

crhoms = AR Term in the Taylor Rule Shock

crhopinf = AR Term in Inflation Rate Shock

crhow = AR Term in Wage Shock

cmap = Price Markup Shock to Inflation Moving Average

cmaw = Price Markup Shock to Wage Moving Average

constepinf =Trend Inflation Growth Rate

constebeta = Constant Inverse Discount Rate

constelab = Steady State Hours Worked

ctrend = Common GDP, Consumption, Investment, and Wages Trend

ea =Stdev of Productivity Shock

eb =Stdev of Risk Premium Shocks

eg = Stdev of Government Spending Shock

eqs =Stdev of Investment Shock

em =Stdev of Taylor Rule Shock

epinf = Stdev of Inflation Shock

ew = Stdev of Wage Shock

First, a brief discussion of the overview of the model. The model is a Representative Agent New Keynesian model with stickiness in both wages and prices. The model has a flexible price set of equations which determines potential GDP for monetary policy and a sticky price system with a price markup equation that determines the true economy. I will only be discussing the sticky price system of equations with an implicit understanding that there is a mirroring flexible price system. Shocks all have auto-regressive components to more accurately match data. All equations

are linearized.

The aggregate resource constraint is:

$$y_t = ccy * c_t + ciy * i_t + \epsilon_t^g + 1 * crkky * z_t \tag{43}$$

The Euler equation is given by:

$$\begin{aligned} c_t =& (chabb/cgamma)/(1 + chabb/cgamma) * c_{t-1} + (1/(1 + chabb/cgamma)) * c_{t+1} + \\ & ((csigma - 1) * cwhlc/(csigma * (1 + chabb/cgamma))) * (l_t - l_{t+1}) - \\ & (1 - chabb/cgamma)/(csigma * (1 + chabb/cgamma)) * (r_t - \pi_{t+1}) + \epsilon_t^b \end{aligned} \tag{44}$$

The investment Euler equations are:

(45)

$$i_t = (1/(1+cbetabar*cgamma))*(i_{t-1}+cbetabar*cgamma*i_{t+1}+(1/(cgamma^2*csadjcost))*q_t)+\epsilon_t^i$$

The arbitrage equation for the value of the capital stock is:

$$\begin{aligned} q_t =& - r_t + \pi_{t+1} + (1/((1 - chabb/cgamma)/(csigma * (1 + chabb/cgamma)))) * \epsilon_t^b + \\ & (crk/(crk + (1 - ctou))) * r_{t+1}^k + ((1 - ctou)/(crk + (1 - ctou))) * q_{t+1} \end{aligned} \tag{46}$$

The linearized production function is:

$$y_t = cfc * (calfa * k_t + (1 - calfa) * l_t + \epsilon_t^a) \tag{47}$$

Capital utilization:

$$k_t^s = k_{t-1} + z_t \tag{48}$$

Capital utilization is some function of the interest rate for capital:

$$z_t = (1/(czcap/(1 - czcap))) * r_t^k \tag{49}$$

Capital's law of motion is:

$$k_t = (1 - cikbar) * k_{t-1} + cikbar * i_t + cikbar * cgamma^2 * csadjcost * \epsilon_t^i \tag{50}$$

The markup equation is:

$$\mu_t^p = calfa * k_t^s + (1 - calfa) * w_t - \epsilon_t^a \tag{51}$$

This equation does not have a flexible price counterpart. Note that the observed variable equations also will not have a flexible price counterpart.

The New Keynesian Phillips curve:

(52)

$$\begin{aligned}\pi_t =& (1/(1 + cbetabar * cgamma * cindp)) * (cbetabar * cgamma * \pi_{t+1} + cindp * \pi_{t-1} + \\ & ((1 - cprobp) * (1 - cbetabar * cgamma * cprobp)/cprobp)/((cfc - 1) * curvp + 1) * (\mu_t^p)) + \\ & \epsilon_t^p\end{aligned}$$

The rental rate of capital equation:

$$r_t^k = w_t + l_t - k_t \tag{53}$$

The path of wages is:

(54)

$$w_t = (1/(1 + cbetabar * cgamma)) * w_{t-1} + (cbetabar * cgamma/(1 + cbetabar * cgamma)) * w_{t+1} +$$
$$(cindw/(1 + cbetabar * cgamma)) * \pi_{t-1} - (1 + cbetabar * cgamma * cindw)/(1 + cbetabar * cgamma) * \pi +$$
$$(cbetabar * cgamma)/(1 + cbetabar * cgamma) * \pi_{t+1} +$$
$$(1 - cprobw) * (1 - cbetabar * cgamma * cprobw)/((1 + cbetabar * cgamma) * cprobw) *$$
$$(1/((clandaw - 1) * curvw + 1)) * (csigl * l_t + (1/(1 - chabb/cgamma)) * c_t -$$
$$((chabb/cgamma)/(1 - chabb/cgamma)) * c_{t-1} - w_t) + \epsilon_t^w$$

The Taylor rule followed by central banks is:

(55) $$r_t = crpi * (1 - crr) * \pi_t + cry * (1 - crr) * (y_t - y_t^p) + crdy * (y_t - y_t^p - y_{t-1} + y_{t-1}^p) + crr * r_{t-1} + \epsilon_t^r$$

The observed variables equations are:

(56) $$dy_t = y_t - y_{t-1} + ctrend$$

(57) $$dc_t = c_t - c_{t-1} + ctrend$$

(58) $$di_t = i_t - i_{t-1} + ctrend$$

(59) $$dw_t = w_t - w_{t-1} + ctrend$$

(60) $$\pi_t^o = \pi_t + constepinf$$

(61) $$r_t^o = r_t + conster$$

(62) $$l_t^o = l_t + constelab$$

The autoregressive equations for shocks are:

$$\epsilon_t^a = crhoa * \epsilon_{t-1}^a + ea \tag{63}$$

$$\epsilon_t^b = crhob * \epsilon_{t-1}^b + eb \tag{64}$$

$$\epsilon_t^g = crhog * \epsilon_{t-1}^g + eg + cgy * ea \tag{65}$$

$$\epsilon_t^i = crhoqs * \epsilon_{t-1}^i + eqs \tag{66}$$

$$\epsilon_t^r = crhoms * \epsilon_{t-1}^r + em \tag{67}$$

$$\epsilon_t^p = crhopinf * \epsilon_{t-1}^p + epinfma - cmap * epinfma(-1) \tag{68}$$

$$epinfma = epinf \tag{69}$$

$$\epsilon_t^w = crhow * \epsilon_{t-1}^w + ewma - cmaw * ewma(-1) \tag{70}$$

$$ewma = ew \tag{71}$$

Some of the higher level parameters in the above equations are also related to the parameters in my diagram by the following equations:

$$cpie = 1 + constepinf/100 \tag{72}$$

$$cgamma = 1 + ctrend/100 \tag{73}$$

$$cbeta = 1/(1 + constebeta/100) \tag{74}$$

$$clandap = cfc \tag{75}$$

$$cbetabar = cbeta * cgamma^{(-csigma)} \tag{76}$$

$$cr = cpie/(cbeta * cgamma^{(-csigma)}) \tag{77}$$

$$crk = (cbeta^{(-1)}) * (cgamma^{csigma}) - (1 - ctou) \tag{78}$$

$$cw = \frac{(calfa^{calfa} * (1 - calfa)^{(1-calfa)}}{(clandap * crk^{calfa}))^{(1/(1-calfa))}} \tag{79}$$

$$cikbar = (1 - (1 - ctou)/cgamma) \tag{80}$$

$$cik = (1 - (1 - ctou)/cgamma) * cgamma \tag{81}$$

$$clk = ((1 - calfa)/calfa) * (crk/cw) \tag{82}$$

$$cky = cfc * (clk)^{(calfa-1)} \tag{83}$$

$$ciy = cik * cky \tag{84}$$

$$ccy = 1 - cg - cik * cky \tag{85}$$

$$crkky = crk * cky \tag{86}$$

$$cwhlc = (1/clandaw) * (1 - calfa)/calfa * crk * cky/ccy \tag{87}$$

$$cwly = 1 - crk * cky \tag{88}$$

$$conster = (cr - 1) * 100 \tag{89}$$

The seven observed equations dy_t, dc_t, di_t, dw_t, π_t^o, r_t^o, l_t^o, are matched using data simulated from a model with true parameters equal to the Smets and Wouters (2007) mean parameters, with slight modifications as some parameters were calibrated to 0. The MCMC approach uses the Smets and Wouters (2007) Dynare code with small modifications like the use of a uniform prior. Further

results are shown in Section IV.F. in the body of the paper.

Here are the 36 parameters estimated by SNPE:

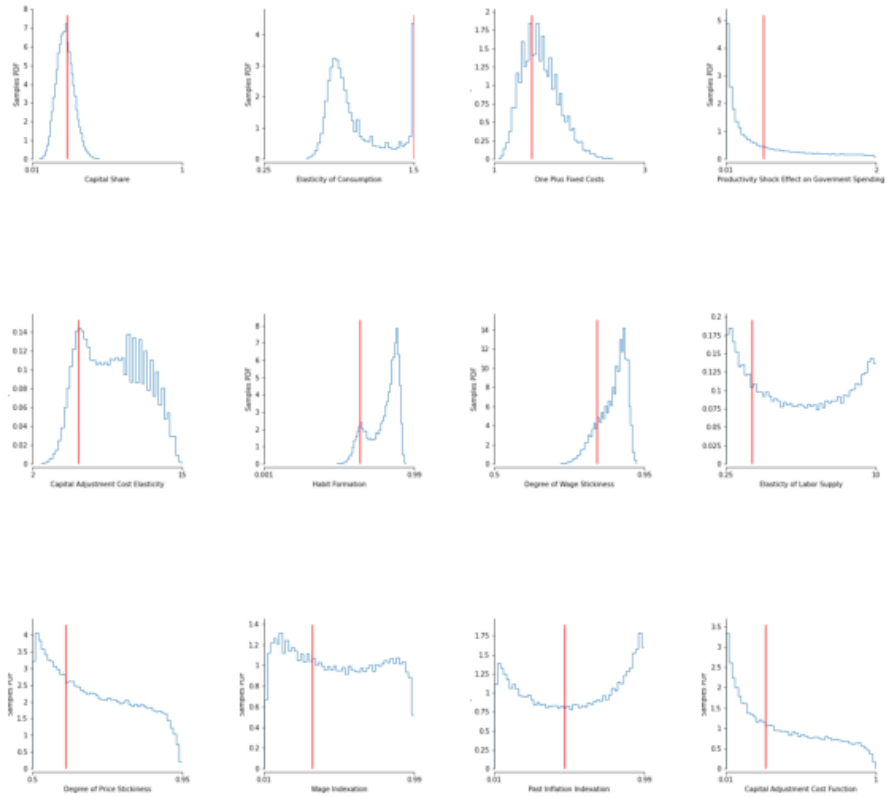

Figure XXIV: Smets-Wouters Posterior with Parameters 1-12

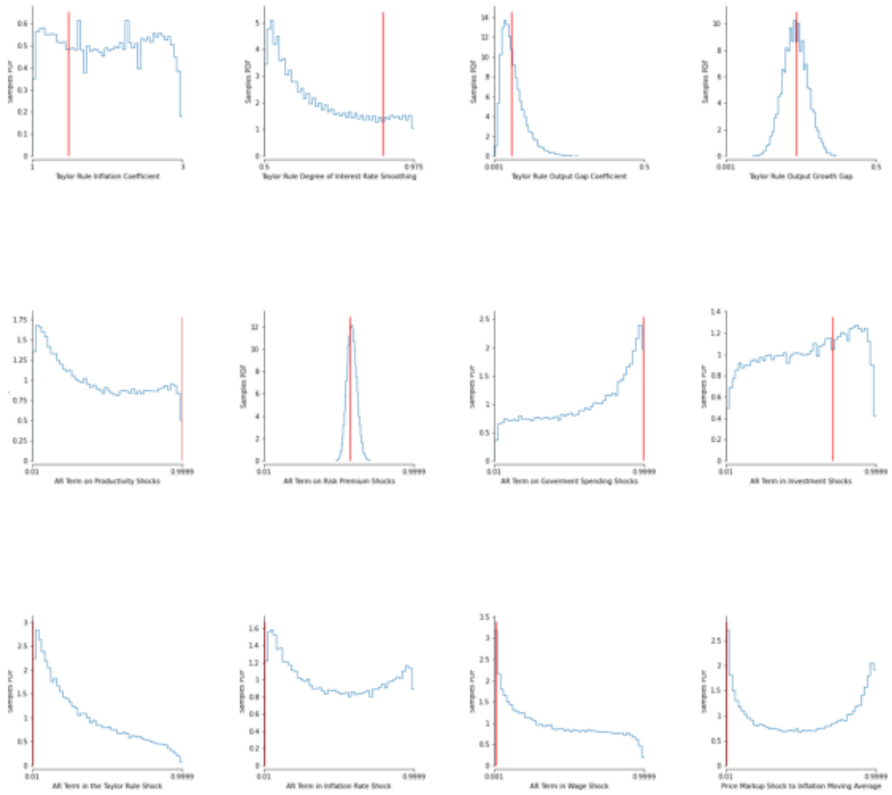

Figure XXV: Smets-Wouters Posterior with Parameters 13-24

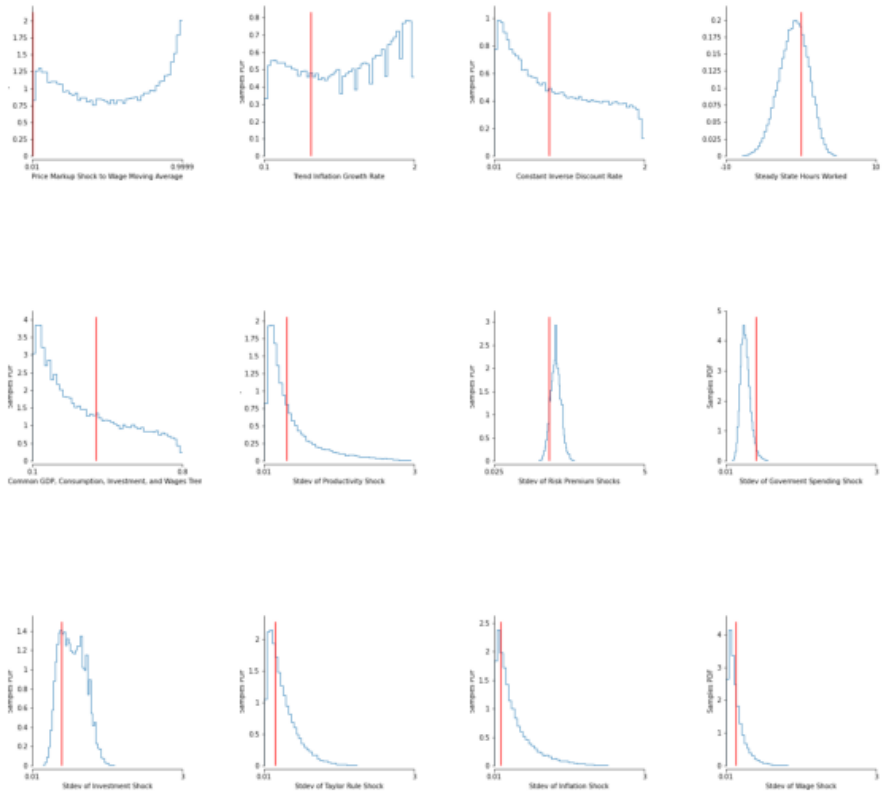

Figure XXVI: Smets-Wouters Posterior with Parameters 25-36

In comparison, here are the 36 parameters estimated via MCMC:

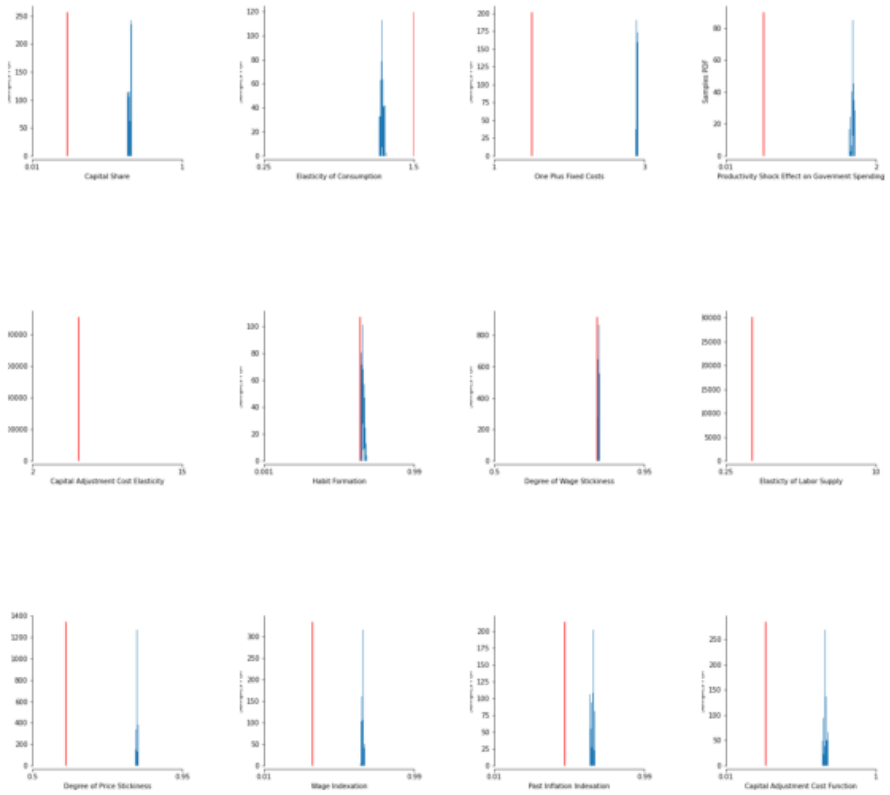

Figure XXVII: Smets-Wouters MH-MCMC Posterior with Parameters 1-12

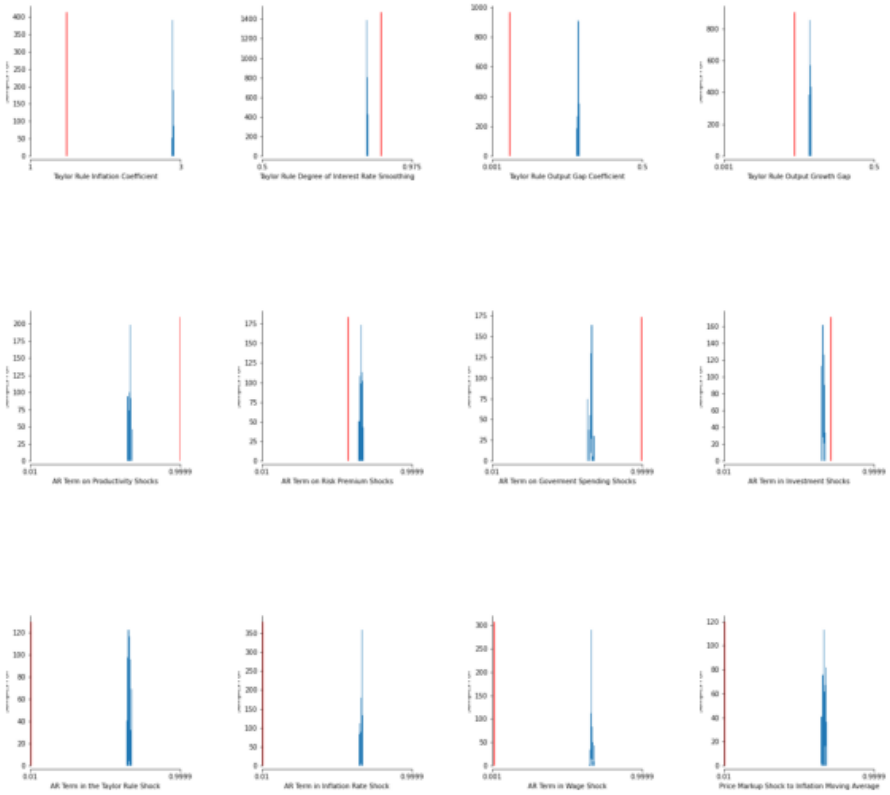

Figure XXVIII: Smets-Wouters MH-MCMC Posterior with Parameters 13-24

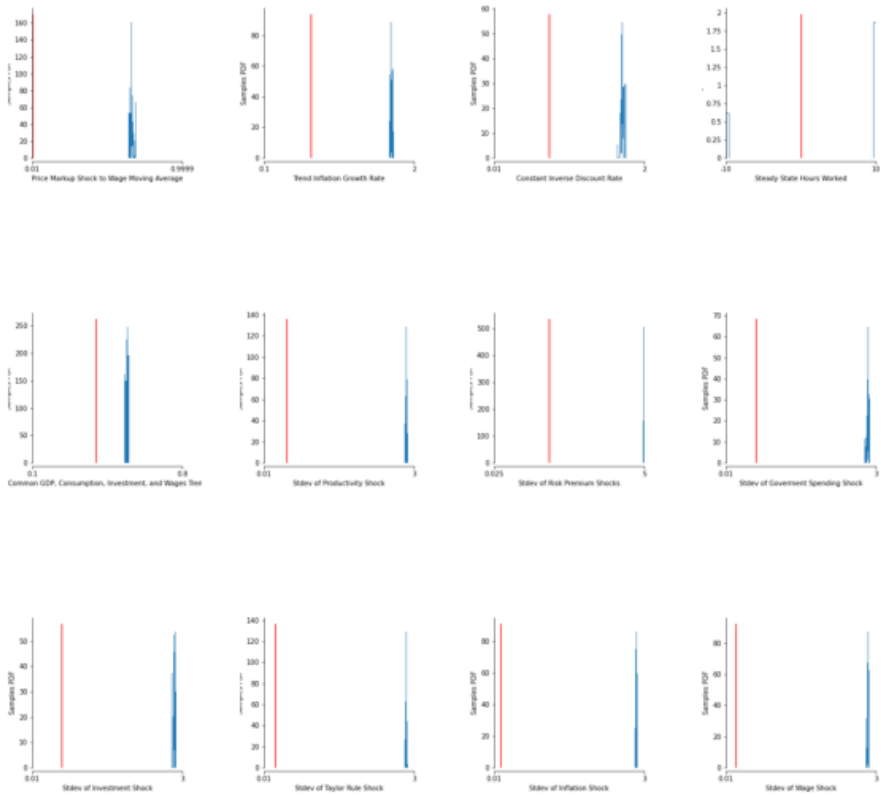

Figure XXIX: Smets-Wouters MH-MCMC Posterior with Parameters 25-36

Finally here are the posteriors estimated by SNPE over only 150 thousand iterations to match the computational time to estimate 10 million MCMC samples:

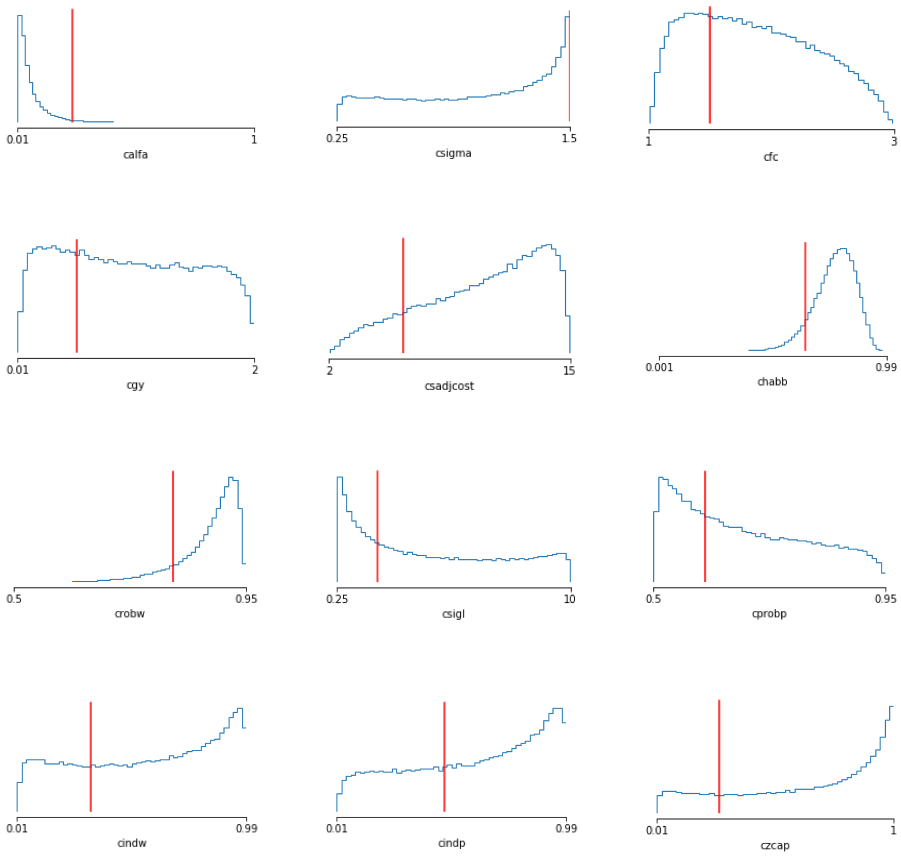

Figure XXX: Smets-Wouters SNPE Posterior with 150 Thousand Samples: Parameters 1-12

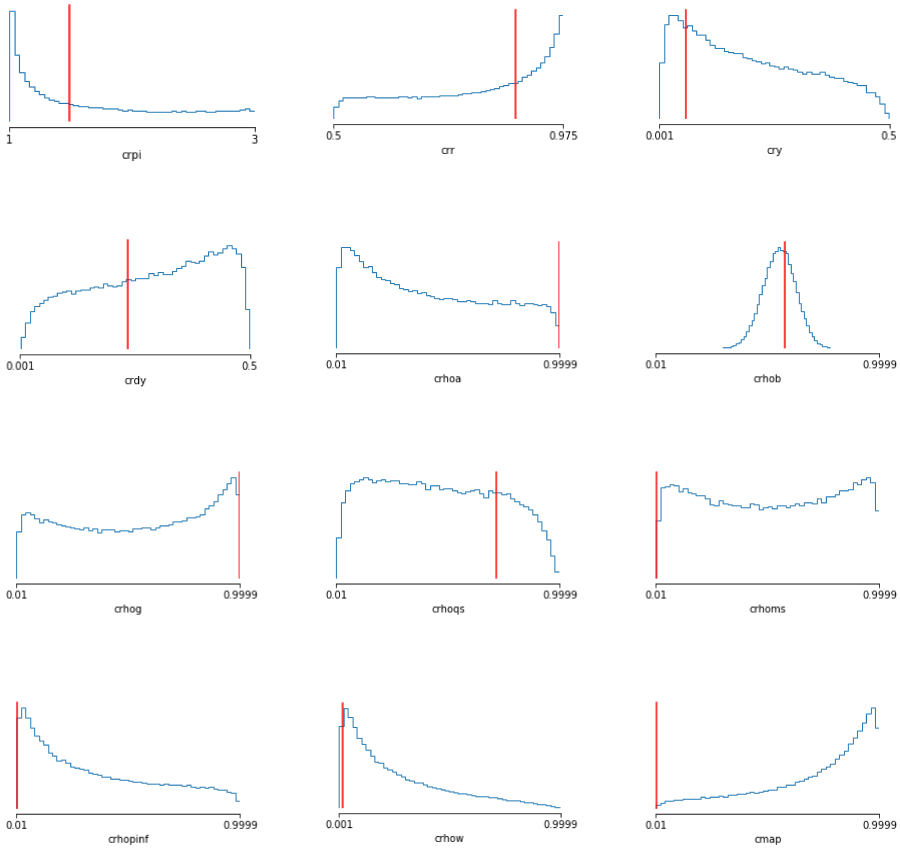

Figure XXXI: Smets-Wouters SNPE Posterior with 150 Thousand Samples: Parameters 13-24

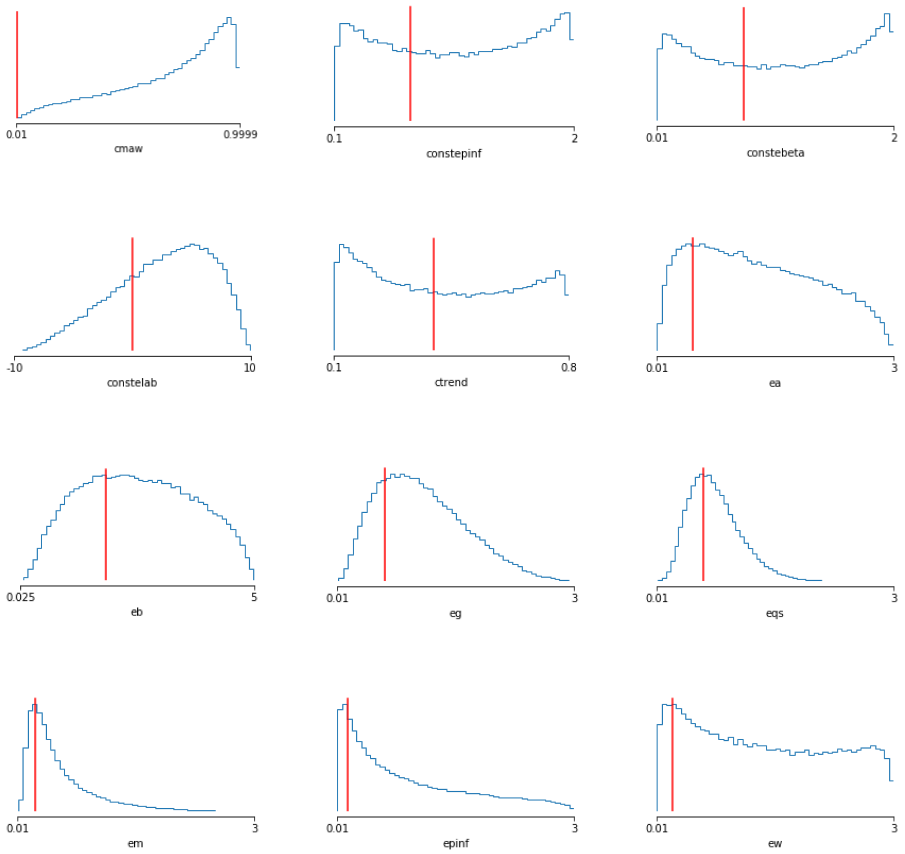

Figure XXXII: Smets-Wouters SNPE Posterior with 150 Thousand Samples: 25-36

BF. The HANK Reiter Model

I estimate two different HANK models. This one is solved via Reiter's method. This provides an example of estimation of models pushing the frontier in economics. The benefits of this approach allows one to estimate heterogeneous agent models in a likelihood manner without resorting to demensionality reduction. The HANK model also comes from Alisdair McKay's tutorial, which adds a heterogeneous distribution of wealth as a state variable for households combined with a relatively standard New Keynesian model.

Preferences are still CRRA, like the RBC model. The firm side is now new Keynesian with

intermediate goods produced via:

$$y_{j,t} = Z_t N_{j,t} \tag{90}$$

Here Z_t is aggregate productivity and N_t is labor employed by firm j producing variety $y_{j,t}$. The final good is produced via the Dixit-Stiglitz aggregator Dixit and Stiglitz (1977):

$$Y_t = (\int_0^1 y_{j,t}{}^{\frac{\epsilon-1}{\epsilon}})^{\frac{\epsilon}{\epsilon-1}} \tag{91}$$

Here ϵ is the elasticity of substitution between intermediates. Output is defined as:

$$Y_t = A_t \int_0^1 n_{j,t} dj = A_t N_t \tag{92}$$

Where $n_{j,t}$ is the labor for the individual intermediate goods. Additionally from the accounting identity, output is also, $Y_t = C_t + \psi M_t H_t$ where the second term is hiring costs.

Government and bonds are defined by the following nominal and real budget constraint equations:

$$\tau P_t(w_t + d_t)(1 - u_t) + P_t B = (1 + i_{t-1})P_{t-1}B + P_t b u_t \tag{93}$$

$$\tau(w_t + d_t)(1 - u_t) + B = R_t B + b u_t \tag{94}$$

Labor market is a standard McCall Search model:

$$N_t = (1-\delta)N_{t-1} + H_t \tag{95}$$

$$1 - u_t = (1-\delta)(1-u_{t-1}) + H_t \tag{96}$$

$$M_t = \frac{H_t}{u_{t-1} + \delta N_{t-1}} \tag{97}$$

$$M_t = \frac{1 - u_t - (1-\delta)(1-u_{t-1})}{u_{t-1} + \delta(1-u_{t-1})} \tag{98}$$

The household maximizes the CRRA utility like the RBC model. The households budget constraint in real money is:

$$a_t + C_t = R_t * a_{t-1} + earnings \tag{99}$$

Here a_t is savings and R_t is the interest rate as determined by the bond market clearing condition. Earnings are given by:

$$\text{earnings} = \begin{cases} (1-\tau_t)\,(w_t + d_t) & \text{if employed} \\ b & \text{if unemployed,} \end{cases} \tag{100}$$

Here w_t is wages, d_t is dividend income and b is unemployment benefits.

The stochastic matrix governing employment, unemployment transitions is given by:

$$\begin{pmatrix} 1 - M_t & \delta \\ M_t & 1-\delta \end{pmatrix} \tag{101}$$

Again, M_t is governed by the labor market equations above.

The firm solves the Dixit-Stiglitz cost problem:

$$y_{j,t} = \left(\frac{p_{j,t}}{P_t}\right)^{-\varepsilon} Y_t \tag{102}$$

with price index P_t given by

$$P_t^{1-\varepsilon} = \int_0^1 p_{j,t}^{1-\varepsilon} dj. \tag{103}$$

The Calvo pricing adjustment parameter θ implies that the intermediate firm chooses price p^* to maximize:

$$\mathbb{E}_t \sum_{s=t}^{\infty} \theta^{s-t} R_{t,s}^{-1} \left[\frac{p_t^*}{P_s} y_{j,s} - w_s n_{j,s} - \psi M_s h_{j,s} \right] \tag{104}$$

Here $n_{j,t}$ denotes employment at firm j in time t. $h_{j,t}$ denotes hiring and $R_{t,s}^{-1}$ denotes the real interest rate discounted s periods back to time t. This model is solved using Reiter's method in combination with time iteration. This model matches unemployment and interest rate to data generated by a parameterized model.

BG. Winberry Approach to Krusell-Smith

The second Heterogeneous Agent model is Krusell and Smith (1998) solved with Winberry (2018), which is presented in the results and also the basis for my empirical application. The consumer is standard:

$$E[\sum_t = 0^{\infty} \beta^t U(c_t) \tag{105}$$

Utility function is CRRA:

$$U(c-t) = \frac{c^{1-\sigma} - 1}{1-\sigma} \tag{106}$$

The investment equation is:

$$y = c + k' - (1-\delta)k \tag{107}$$

Agents can be employed or unemployed. If employed, it works one unit of labor or else it does not work. This is denoted with an ϵ. Employment status transitions via a Markov matrix.

On the market side, wages and rental rate equal their marginal benefit. In particular if aggregating across all capital ($\bar{k}$) and labor ($\bar{l}$):

$$w = (1-\alpha)z(\bar{k}/\bar{l})^{\alpha} r = \alpha z(\bar{k}/\bar{l})^{1-\alpha} \tag{108}$$

The states are the distribution of capital and labor, denoted Γ, as well as productivity. For each individual, they optimize via:

$$v(k,\epsilon;\Gamma,z) = max_{c,k'}\{U(c) + \beta E[v(k,\epsilon;\Gamma,z)|z,\epsilon]\} \tag{109}$$

subject to

$$c + k' = rk + wl\epsilon + (1-\delta)k\Gamma' = H(\Gamma,z,z')k' \geq 0 \tag{110}$$

Each agent optimizes according to their own k and ϵ. However the prices are chosen so that when agents are optimizing, the aggregate next period levels of k and ϵ match the levels implied by the prices. As such you are solving for a fixed point so that given a distribution of k and ϵ, the next period distribution of k and ϵ also correspond to the prices of k and ϵ that would induce agents to acquire those amounts. Winberry takes a different approach. Since you know the individual's law of motion and you can integrate the distribution to get the aggregate prices that generate that law of motion, you can derive a law of motion for the heterogeneous distribution. If you approximate this distribution via moments, you can derive a law of motion for the moments of the distribution. Then you can apply non-heterogeneous agent approaches to solve this model. In particular, Winberry solves the model in Dynare with perturbation. For more details on the solution method, see Winberry (2018).

Liu and Plagborg-Møller (2023) perform a likelihood estimation on a Krusell-Smith model solved via Winberry. Estimating full information heterogeneous agent models via a likelihood is difficult

for models solved via Winberry's approach. The reason is that when evaluating the micro-data one has to calculate the likelihood of the the micro data conditional on the distribution of the state and macro variables. One does this by sampling from the Kalman filter state-space, macro variables conditioned on the state, and micro variables conditioned on the macro variables and the state. If you need n points to have good coverage of a distribution, you will need n^3 samples to have good coverage. Due to the fact that SNPE does not require likelihood calculation, this whole process is avoided.

C Background on Simulation Neural Ratio Estimators

CA. General Approach of Simulation-Based Inference

The differences between the two simulation-based inference approaches roughly centers around what conditional density estimator to use. Sequential Neural Ration Estimators (SNRE) uses a GAN while Sequential Neural Posterior Estimators (SNPE) uses a Normalizing Flow or Conditional Mixture of Gaussians. While one can sample a lot of points from the joint distribution, an iterative approach where one fits the density estimator on sampled data then re-samples leads to lower variance. I will start discussing SNRE, by introducing the GAN

CB. Generative Adversarial Networks (GAN)

While I focus on SNPE, there are related approaches that use a GAN (Goodfellow et al., 2014) as a density estimator. For example, SNRE, and another approach built on SNRE, SNVI, can be used. The GAN is an algorithm that uses two competing models moving in tandem to generate data that matches data generated from the real world. Below is a diagram describing the structure of a basic GAN:

The generator is a neural network (or any differentiable model like a dynamic structural model) which attempts to produce data (unconditionally or conditionally via some inputs Z) that resembles data in the real world. The discriminator is another neural network (or any trainable model that

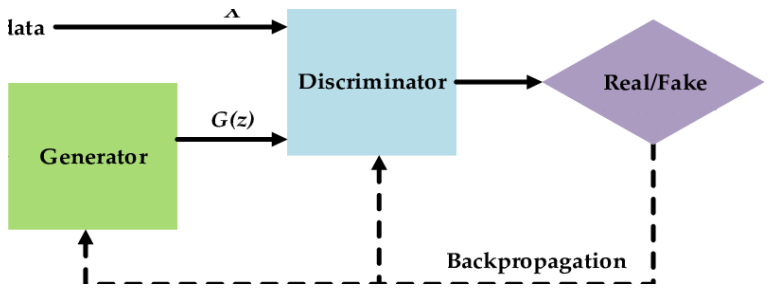

FIGURE XXXIII: DIAGRAM OF A GAN

can produce a logistic output), which outputs the probability a given input data comes from the underlying data generating process and not from the generator. The two networks play a game where the objective of the generator is to convince the discriminator to assign high likelihood to its output and the objective of the discriminator is to assign the correct probabilistic label, a number between 0 or 1, to input that represents the continuum that the current data point is generated data or real data respectively. One way to think about a GAN is as an MSM algorithm where the moment conditions automatically evolve over many rounds to pick the most mismatched moments between the true data and the model. When thought in this way, it is intuitive to recognize that the GAN is nearly a full information and efficient estimation procedure. For more thorough proofs, see Kaji, Manresa and Pouliot (2020).

Mathematically the discriminator loss is:

$$Loss_D = -E_{x \sim P_{data}(x)}[D(x)] - E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{111}$$

This is just a logit loss applied to data either coming from the data generating process or the generator. Likewise, the generator's loss is to fool the discriminator:

$$Loss_G = E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{112}$$

These two loss functions can be compactly represented as:

$$G, D = \min_{G} \max_{D} E_{x \sim P_{data}(x)}[D(x)] + E_{z \sim P_{sim}(z)}[1 - D(G(z))] \tag{113}$$

Equation (113) makes it clear the GAN objective is a minimax game with a solution that is a Nash equilibrium. However what is important for simulation-based estimation is to recover likelihoods/probabilities. The next proposition shows how to do that:

Appendix Proposition 1.: For a fixed generator, the optimal discriminator will produce a logistic output:

$$D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_{generator}(x)} \tag{114}$$

See Goodfellow et al. (2014) for the proof. Using this, one can use the discriminator output to estimate a likelihood ratio.

CC. Sequential Neural Ratio Estimation

In SNRE, Durkan, Murray and Papamakarios (2020) use a GAN, not to match underlying data like in Appendix Proposition 1, but to contrast between two different distributions to estimate a density. In macro, the data set is typically a single data point with correlated time-steps. Using a GAN to discriminate between real data and fake data would not work because the GAN would learn a delta function for the real data. In order to ameliorate this, the authors use contrastive learning where they differentiate between the generated distribution and the independent distribution, $X, \theta \sim P(X)P(\theta)$, where X is sampled uniformly from the empirical distribution of simulated data, rather than the correct associated θ_i. Then they contrast the independent distribution, with the true joint: $X, \theta \sim p(X, \theta) = p(X|\theta)p(\theta)$. Furthermore, knowing the optimal discriminator gives the ratio discussed in (114). The authors then recover the ratio: $r(X, \theta) = \frac{p(X, \theta)}{P(X)P(\theta)}$, using the fact

that $\frac{p(X,\theta)}{p(X,\theta)+p(X)P(\theta)}$ equals $\frac{r}{r+1}$. Thus, the density of the posterior is $P(\theta|X) = r(X,\theta)P(\theta)$, where $P(\theta)$ is the prior. This extends Kaji, Manresa and Pouliot (2020) GAN approach in allowing estimation of models with a latent time dimension. One can extend this to multi-round inference where one uses the GAN as a better proposal distribution than the prior, by sampling θ from the GAN using MCMC. Then one uses importance sampling to deal with the mismatch in proposal distribution and prior. See Durkan, Murray and Papamakarios (2020), Hermans, Begy and Louppe (2020) and the SNRE theoretical sections of the Appendix for more information. The below shows the steps for performing SNVI:

Algorithm 2: SNRE Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', discriminator $d_\phi(X,\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with MCMC;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X,\theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{X^{c(k)}\}_k^{B*K}, \{\theta^{c(k)}\}_k^{B*K} \sim D\{X\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions X, θ and X^c, θ^c and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(X^{(i)},\theta^{(i)})}{\sum_k^K exp(d_\phi(X^{(k)},\theta^{(k)})})$;

  end

  Update posterior $p^{(i)} \propto exp(d_\phi(X',\theta))p(\theta)$;

end

CD. Sequential Neural Variational Inference (SNVI)

In SNVI, Glöckler, Deistler and Macke (n.d.), one performs SNRE, but instead of MCMC sampling from the GAN posterior, one projects the distribution derived by the GAN unto a Normalizing Flow with Variational Inference. As discussed in this article, MCMC takes a long time to converge to the posterior. Since one can calculate the pdfs of a sample point using the GAN ratio estimation, one can sample from a Normalizing Flow, calculate the pdf of the sample under both the flow and the GAN, and use this to minimize the KL divergence between the flow and the GAN. Thus the flow inherits the distribution of the GAN, and one can sample directly from the flow without resorting to MCMC. The downside of this approach is that using the reverse KL divergence, in finite iterations, leads to the learned posterior being too tight around the mode. The followings describes the SNVI step by step:

Algorithm 3: SNVI Algorithm

Input: Simulator $p(X|\theta)$, prior $p(\theta)$, data X', discriminator $d_\phi(X, \theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim f_{\phi'}^{(i-1)}(\theta)$ for $n = 1...S$;

  Simulate $X^{(n)} \sim P(X|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{X^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(X, \theta)$ *not converged* **do**

    Sample $\{X^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Sample K contrasting samples for each i sample $\{X^{c(k)}\}_k^{B*K}, \{\theta^{c(k)}\}_k^{B*K} \sim D\{X\}, D\{\theta\}$, breaking the joint distribution of D into independent marginals;

    Train the GAN using the GAN loss function from the two distributions X, θ and X^c, θ^c and a softmax/multinomial logit objective $\sum_i^B log(\frac{exp(d_\phi(X^{(i)}, \theta^{(i)})}{\sum_k^K exp(d_\phi(X^{(k)}, \theta^{(k)})})$;

  end

  Train Normalizing Flow with Variational Inference $f_{\phi'}^i = argmin_{\phi'}(KL(f_{\phi'}^i(\theta)||exp(d_\phi(X', \theta))p(\theta))$;

end

Department of Economics, University of Michigan, Ann Arbor